



AWS Data Engineering Project



Pratiyasa Pritimayi Sahu
Silicon University
24BCSI21

Project-1
AWS Server less
Data Pipeline

Given by Ingenious Tech World

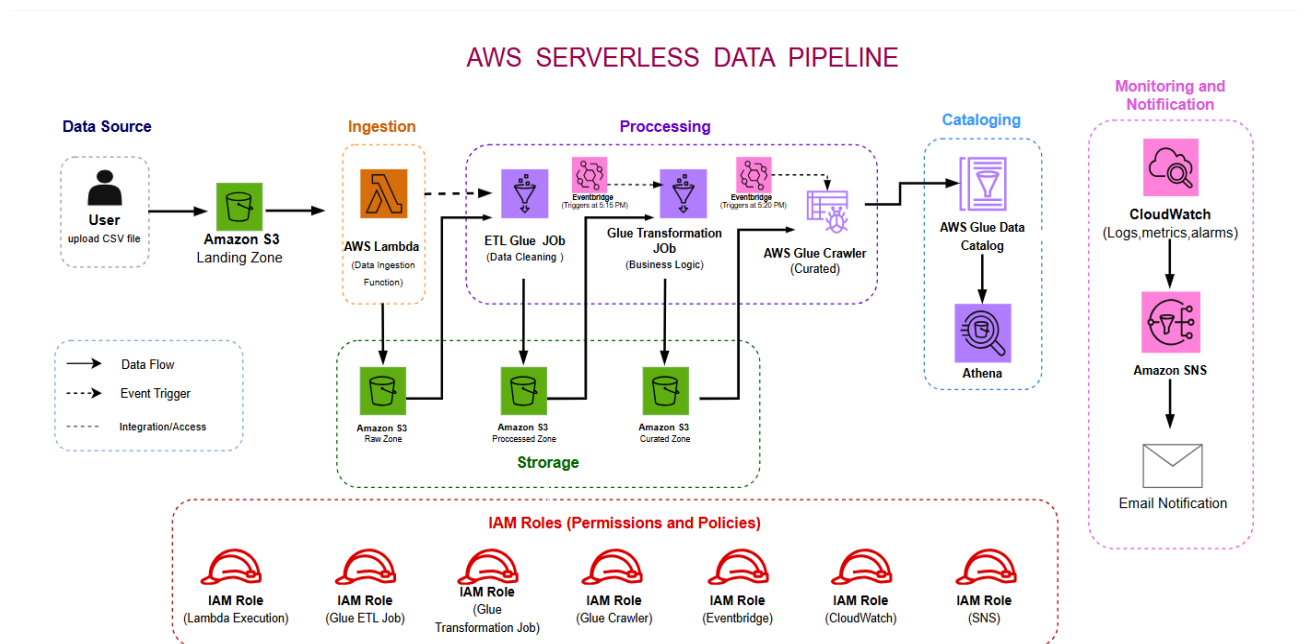
Contents

- 1.Project Overview
- 2.Architecture Diagram
- 3.Architecture Explanation
- 4.Workflow Description
- 5.AWS Services Used
- 6.Source Code
- 7.Step by Step implementation
- 8.Challenges Faced
- 9.Learning Outcome

• Project Overview

The AWS Serverless Sales Data Pipeline is designed to automate the ingestion, processing, transformation, cataloging, and analysis of sales data using fully managed AWS services. Whenever a CSV file is uploaded to the Landing S3 bucket, an AWS Lambda function validates the file and copies it to the Raw S3 bucket. The pipeline then executes an AWS Glue ETL job to clean and process the data into Parquet format, followed by a transformation job that calculates business metrics such as Total Sales, Discount Amount, Final Sales, Order Year, and Order Month. AWS Glue Crawlers update the Glue Data Catalog, enabling Amazon Athena to query the processed data directly from Amazon S3. CloudWatch monitors the pipeline, SNS sends email notifications on failures, and EventBridge Scheduler automates the execution of Glue Crawlers and Transformation Jobs.

• Architecture Diagram



• Architecture Explanation

The architecture follows a serverless event-driven data engineering approach.

1. Sales data is uploaded into the Amazon S3 Landing Bucket.
 2. Amazon S3 triggers an AWS Lambda function.
 3. Lambda validates the uploaded CSV file and copies it to the Raw Bucket.
 4. Lambda starts the AWS Glue Cleaning Job.
 5. The Cleaning Job removes invalid records and stores cleaned data in the Processed Bucket as Parquet files.
 6. The Glue Crawler scans the Processed Bucket and updates the Glue Data Catalog.
 7. The Glue Transformation Job generates business insights and stores curated analytics data in the Curated Bucket.
 8. Another Glue Crawler catalogs the curated dataset.
 9. Amazon Athena queries the curated data for analytics.
 10. Amazon CloudWatch collects logs and metrics.
 11. Amazon SNS sends failure notifications through email.
 12. EventBridge Scheduler automatically runs the Glue Crawler and Transformation Job every day.
-

• Workflow Description

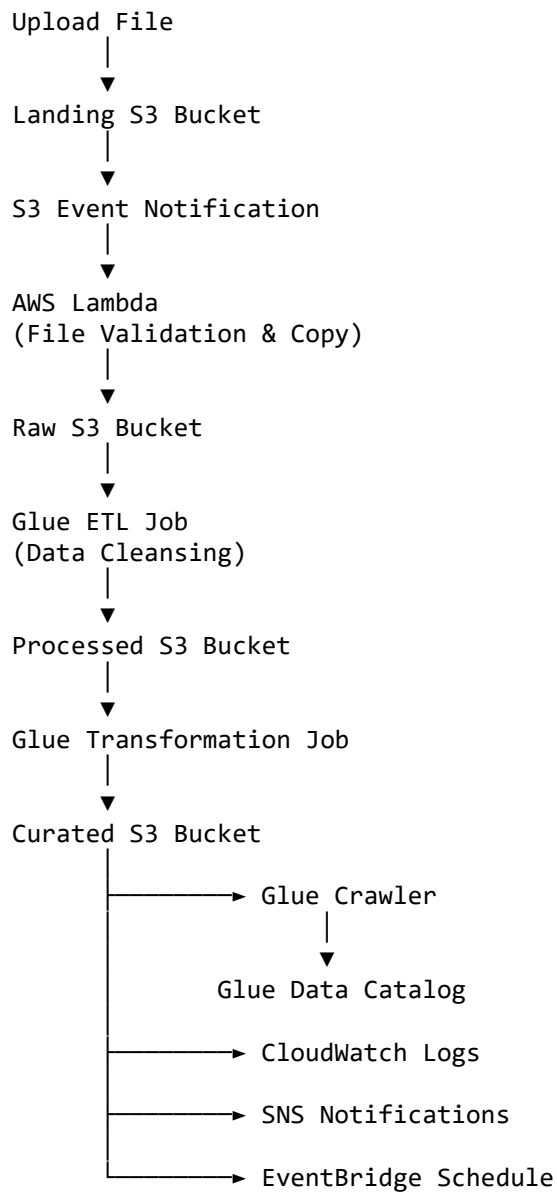
The workflow begins when a CSV sales file is uploaded to the Landing S3 bucket. The upload event automatically invokes an AWS Lambda function, which validates the file type and copies it into the Raw Bucket under the *sales/* folder. After the file is copied successfully, Lambda starts the Glue Cleaning Job.

The Cleaning Job reads the raw sales data, removes duplicate or invalid records, standardizes the schema, and converts the cleaned data into Parquet format before storing it in the Processed Bucket.

A scheduled Glue Crawler scans the Processed Bucket and updates the AWS Glue Data Catalog. After the crawler finishes, the Transformation Job runs automatically through EventBridge Scheduler. This job generates business metrics including Total Sales, Discount Amount, Final Sales, Order Year, Order Month, and Transformation Timestamp before storing the transformed dataset in the Curated Bucket.

A second Glue Crawler updates the Glue Catalog for the curated dataset. Finally, Amazon Athena queries the curated Parquet data for reporting and analysis.

Throughout the pipeline, CloudWatch captures logs while SNS notifies administrators if any component fails.



• AWS Services Used

AWS Service	Purpose
Amazon S3	Stores Landing, Raw, Processed, and Curated datasets

AWS Service	Purpose
AWS Lambda	Validates uploaded files, copies them to Raw Bucket, and triggers the Glue Cleaning Job
AWS Glue ETL	Cleans and transforms sales data
AWS Glue Crawlers	Automatically update the Glue Data Catalog
AWS Glue Data Catalog	Maintains metadata for Athena queries
Amazon Athena	Performs SQL queries on Parquet files stored in S3
Amazon EventBridge Scheduler	Schedules Glue Crawlers and Transformation Jobs
Amazon CloudWatch	Stores logs and monitors Glue and Lambda execution
Amazon SNS	Sends email alerts whenever failures occur
AWS IAM	Provides secure access permissions between AWS services

• Source Code

1. Lambda Function code

```
import json
import boto3
import os
import urllib.parse

# AWS Clients
s3 = boto3.client("s3")
glue = boto3.client("glue")

# Environment Variables
RAW_BUCKET = os.environ["RAW_BUCKET"]

def lambda_handler(event, context):

    # Get uploaded file details
    source_bucket = event["Records"][0]["s3"]["bucket"]["name"]
    source_key = urllib.parse.unquote_plus(
        event["Records"][0]["s3"]["object"]["key"]
    )

    print(f"File Uploaded: {source_key}")
```

```

# Allow only CSV files
if not source_key.lower().endswith(".csv"):
    print("Invalid file type.")

    return {
        "statusCode": 400,
        "body": json.dumps("Only CSV files are allowed.")
    }

# Destination in Raw Bucket
file_name = os.path.basename(source_key)
destination_key = f"sales/{file_name}"

try:

    # Copy file to Raw Bucket
    s3.copy_object(
        Bucket=RAW_BUCKET,
        CopySource={
            "Bucket": source_bucket,
            "Key": source_key
        },
        Key=destination_key
    )

    print(f"Copied to {RAW_BUCKET}/{destination_key}")

    # Start Glue Cleaning Job
    response = glue.start_job_run(
        JobName="SalesDataCleaningJob"
    )

    print(f"Glue Job Started: {response['JobRunId']}")

    return {
        "statusCode": 200,
        "body": json.dumps({
            "message": "File copied successfully.",
            "GlueJobRunId": response["JobRunId"]
        })
    }

except Exception as e:
    print(f"Error: {str(e)}")
    raise e

```

2. Glue ETL Script

```

import sys
from pyspark.sql import SparkSession

```

```

from pyspark.sql.functions import (
    col,
    current_timestamp,
    trim,
    when
)
from pyspark.sql.types import IntegerType, DoubleType

# -----
# Spark Session
# -----

spark = (
    SparkSession.builder
    .appName("SalesDataCleaning")
    .getOrCreate()
)

print("==== ETL JOB STARTED ====")

# -----
# Bucket Paths
# -----

RAW_PATH = "s3://pratiyasa-raw-zone/sales/"

PROCESSED_PATH = "s3://pratiyasa-processed-zone/cleaned/"

try:

    # -----
    # Read CSV
    # -----

    print("Reading CSV...")

    df = (
        spark.read
        .option("header", True)
        .option("inferSchema", False)
        .csv(RAW_PATH)
    )

    print(f"Rows Before Cleaning : {df.count()}")

    # -----
    # Convert Empty Strings to NULL
    # -----

    df = (
        df.withColumn(
            "Customer_Name",
            when(
                trim(col("Customer_Name")) == "",
                None
            ).otherwise(col("Customer_Name"))
        )
    )

```

```

    )
    .withColumn(
        "Quantity",
        when(
            trim(col("Quantity")) == "",
            None
        ).otherwise(col("Quantity"))
    )
)

# -----
# Remove Duplicate Records
# -----

df = df.dropDuplicates()

# -----
# Remove Missing Values
# -----

df = (
    df.filter(col("Customer_Name").isNotNull())
      .filter(col("Quantity").isNotNull())
)

# -----
# Convert Datatypes
# -----

df = (
    df.withColumn(
        "Quantity",
        col("Quantity").cast(IntegerType())
    )
    .withColumn(
        "Unit_Price",
        col("Unit_Price").cast(DoubleType())
    )
    .withColumn(
        "Discount",
        col("Discount").cast(DoubleType())
    )
)

# -----
# Add ETL Timestamp
# -----

df = df.withColumn(
    "ETL_Load_Time",
    current_timestamp()
)

print(f"Rows After Cleaning : {df.count()}")

# -----

```

```

# Write Parquet
# -----

print("Writing Parquet...")

(
    df.write
    .mode("overwrite")
    .option("compression", "snappy")
    .parquet(PROCESSED_PATH)
)

print("==== ETL JOB COMPLETED SUCCESSFULLY =====")

except Exception as e:

    print("ETL FAILED")

    print(str(e))

    raise

```

3. Glue Transformation Script

```

from pyspark.sql import SparkSession
from pyspark.sql.functions import (
    col,
    year,
    month,
    current_timestamp,
    round
)

spark = (
    SparkSession.builder
    .appName("SalesTransformation")
    .getOrCreate()
)

print("==== TRANSFORMATION JOB STARTED =====")

INPUT_PATH = "s3://pratiyasa-processed-zone/cleaned/"
OUTPUT_PATH = "s3://pratiyasa-curated-zone/analytics/"

try:

    df = spark.read.parquet(INPUT_PATH)

    print(f"Rows Read : {df.count()}")

    # Business Columns
    df = df.withColumn(
        "Total_Sales",
        col("Quantity") * col("Unit_Price")
    )

```

```

    )

    df = df.withColumn(
        "Discount_Amount",
        round(
            col("Total_Sales") * col("Discount") / 100,
            2
        )
    )

    df = df.withColumn(
        "Final_Sales",
        col("Total_Sales") - col("Discount_Amount")
    )

    df = df.withColumn(
        "Order_Year",
        year(col("Order_Date"))
    )

    df = df.withColumn(
        "Order_Month",
        month(col("Order_Date"))
    )

    df = df.withColumn(
        "Transformation_Time",
        current_timestamp()
    )

    print("Writing Curated Data...")

    (
        df.write
        .mode("overwrite")
        .partitionBy("Order_Year", "Order_Month")
        .option("compression", "snappy")
        .parquet(OUTPUT_PATH)
    )

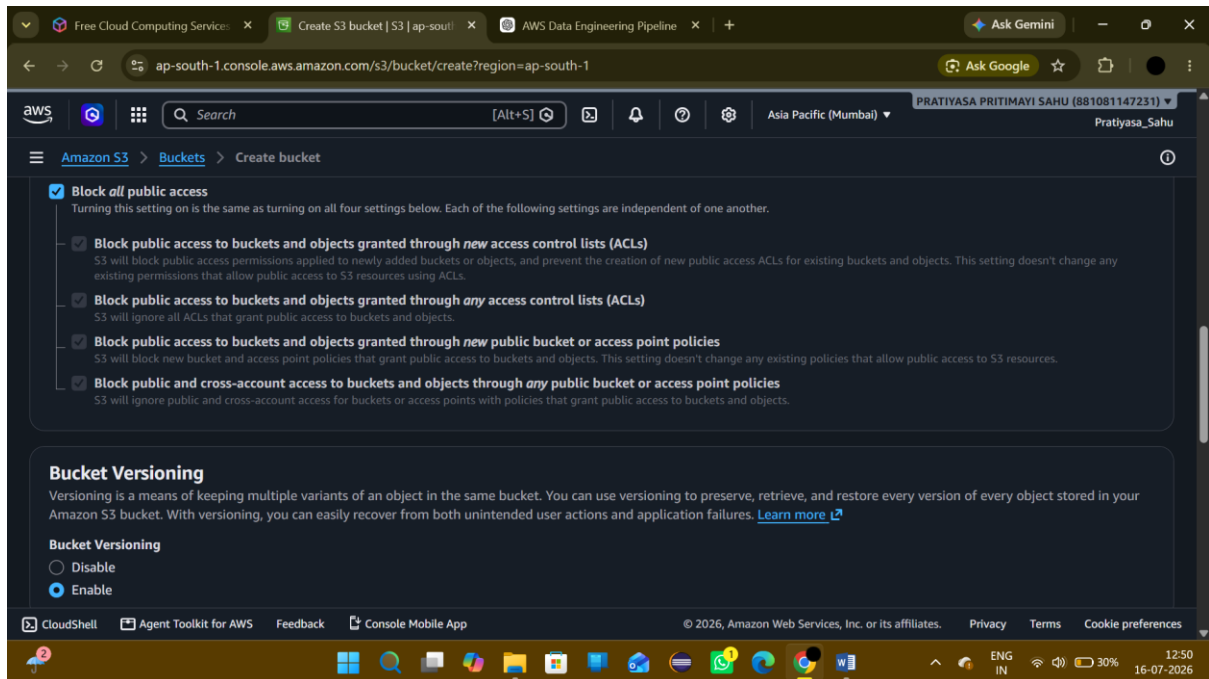
    print("===== TRANSFORMATION COMPLETED =====")

except Exception as e:
    print(str(e))
    raise

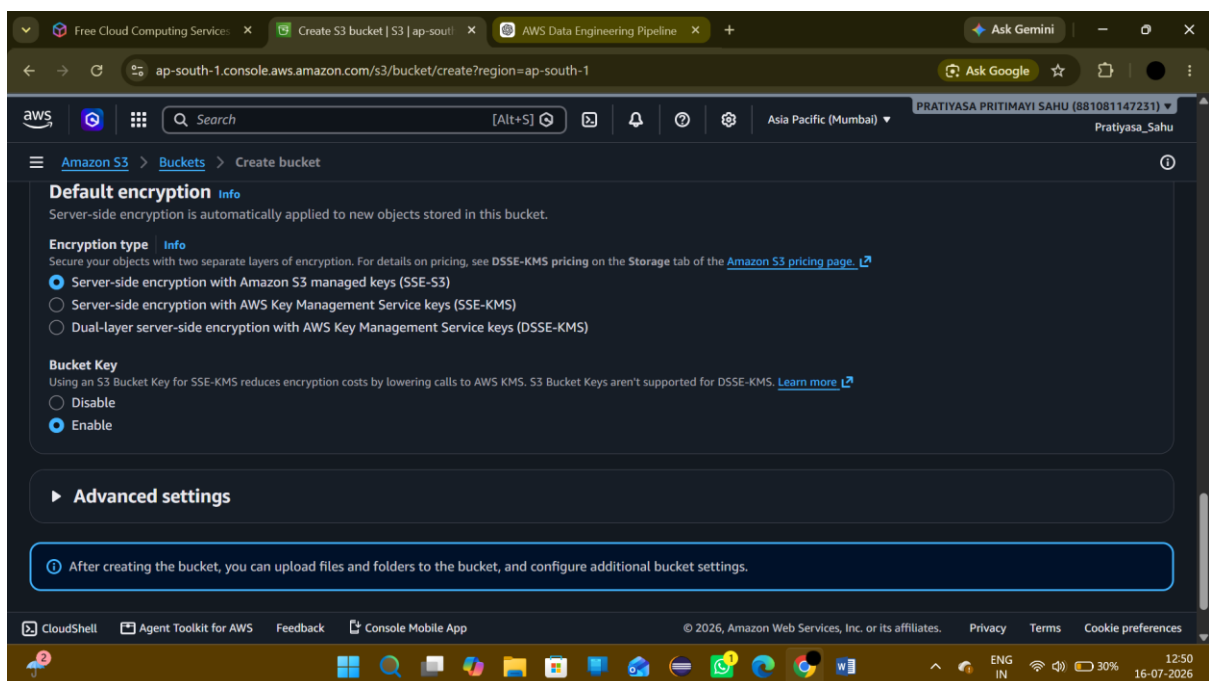
```

STEP-BY-STEP IMPLEMENTATION

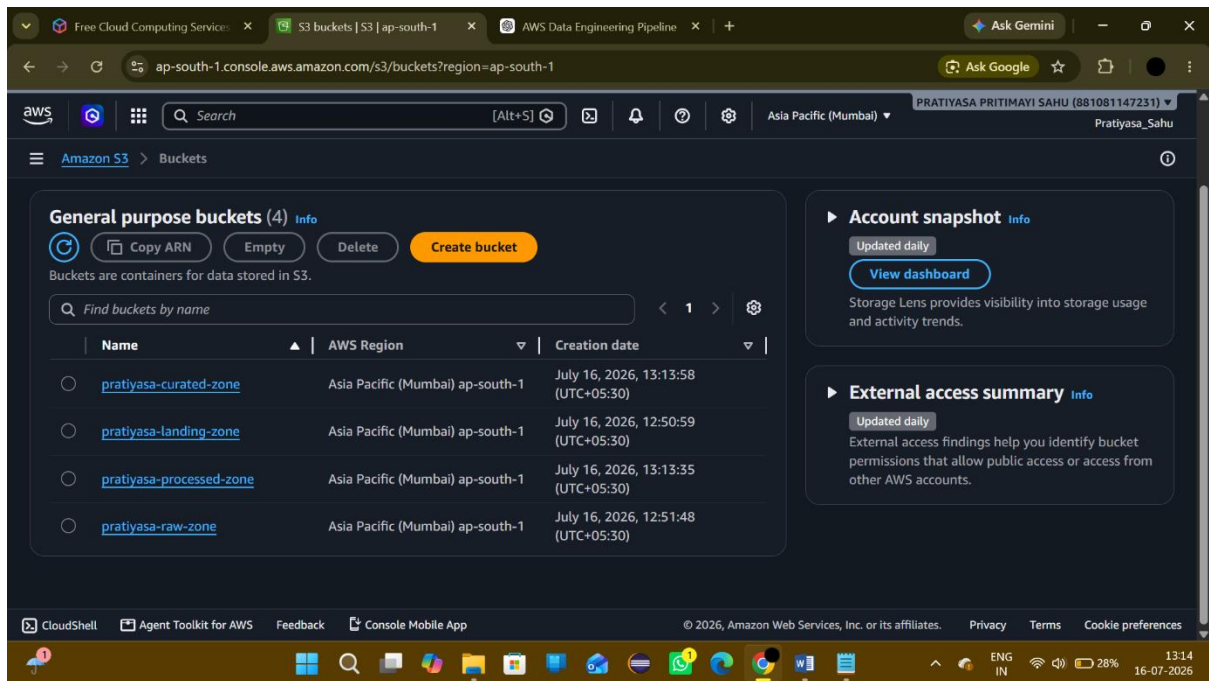
Creating S3 buckets with bucket versioning enabled



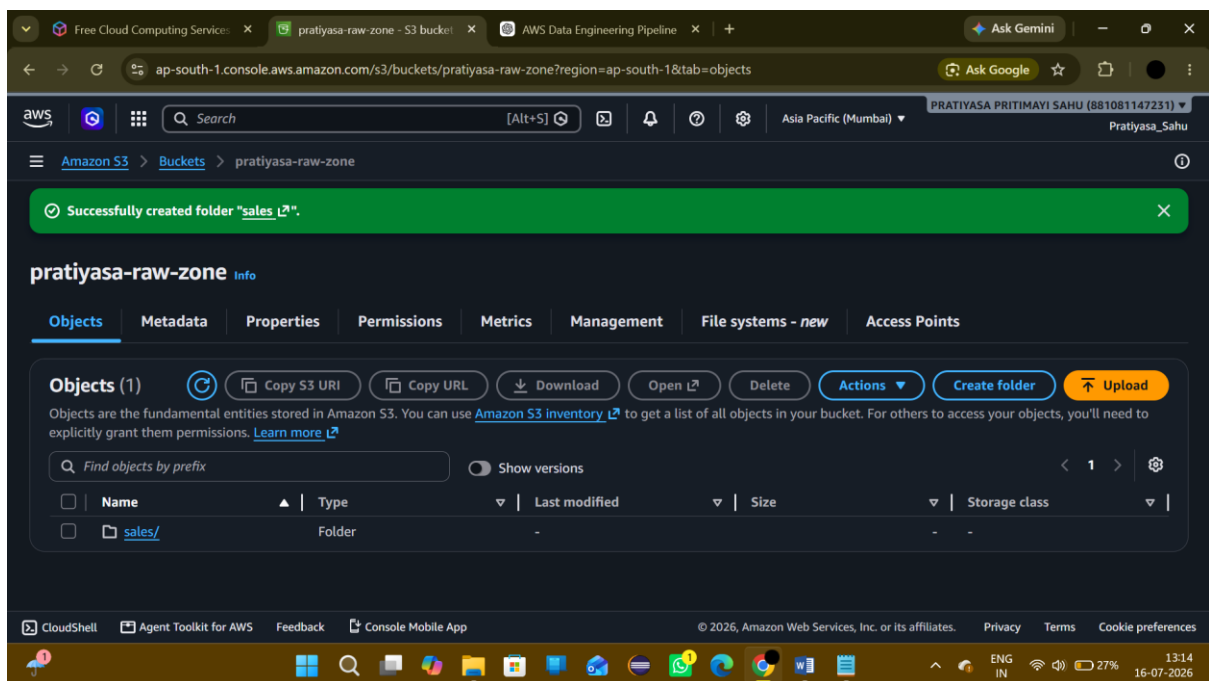
SSE-S3 Encryption type and bucket key enabled



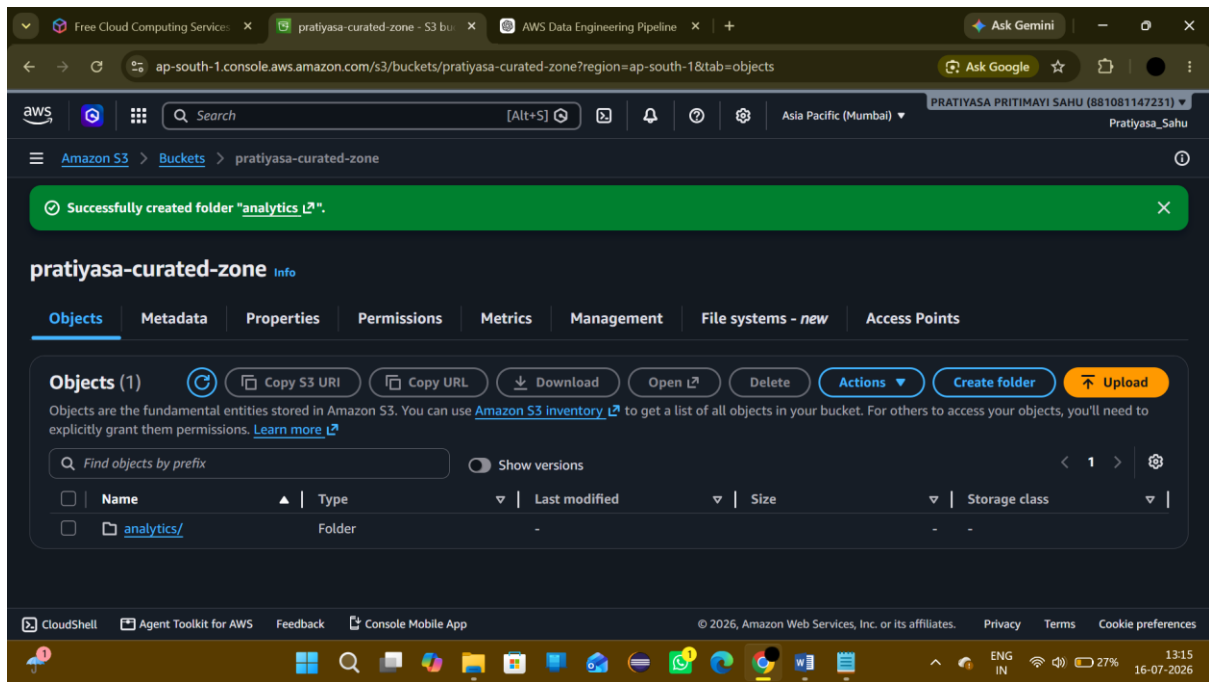
All necessary S3 buckets



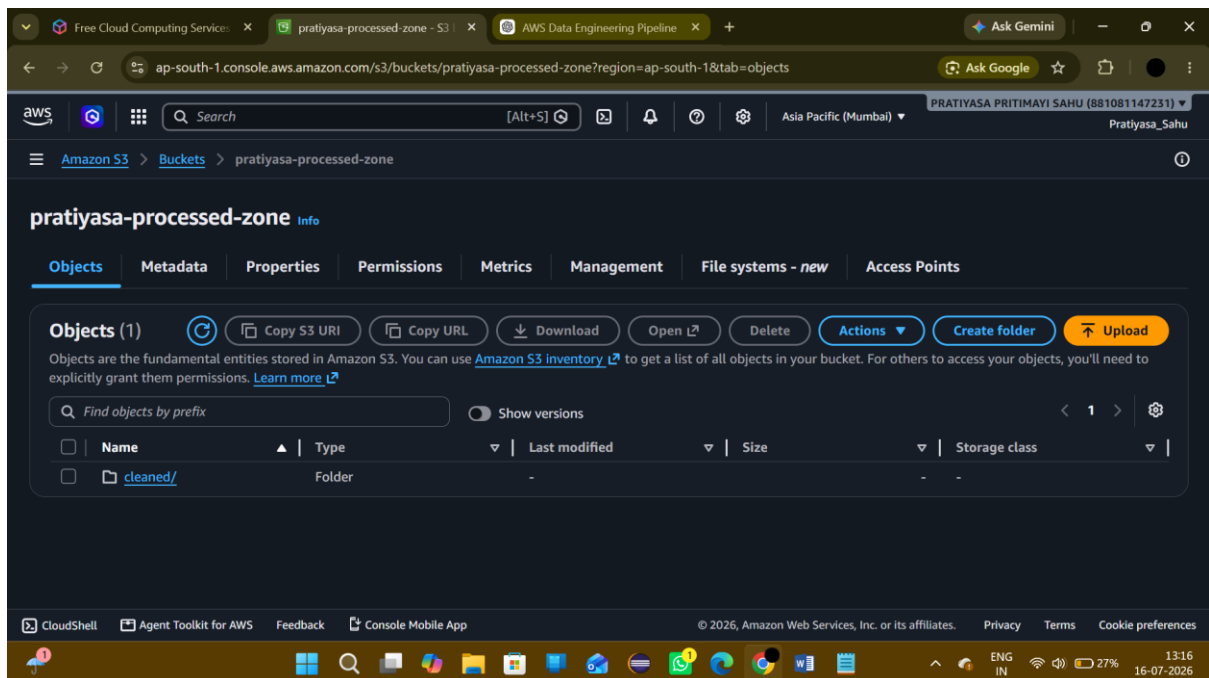
Created sales/ folder in raw-zone bucket



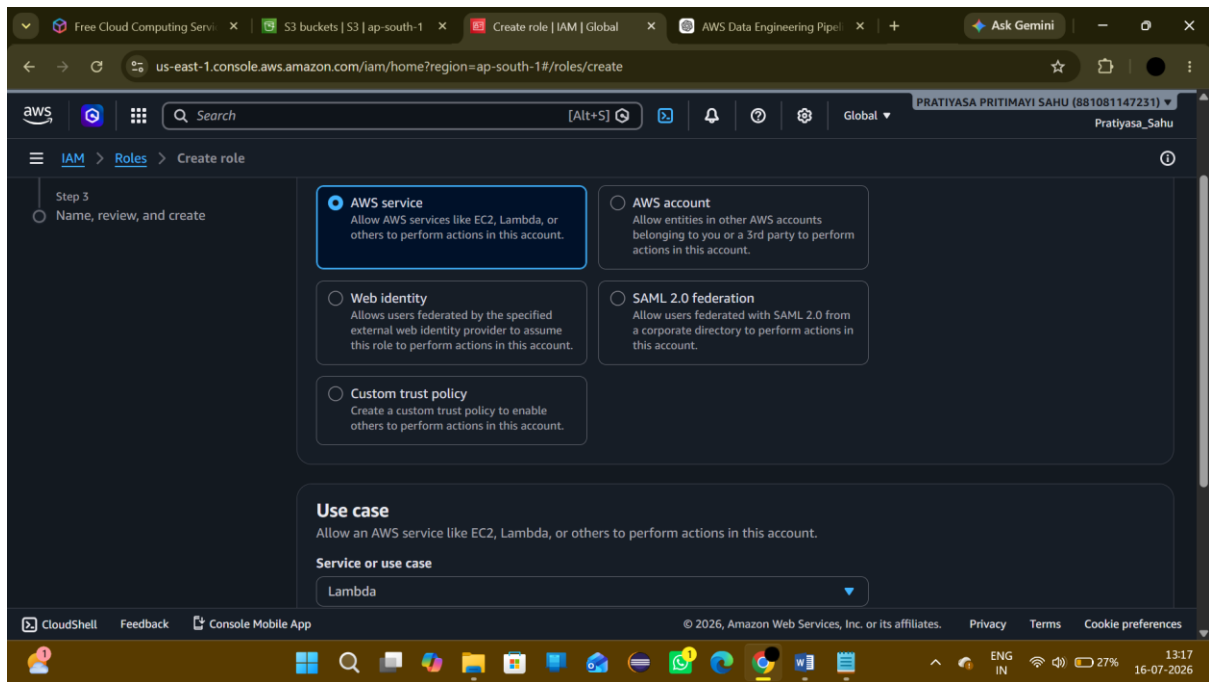
Created analytics/ folder in curated-zone bucket



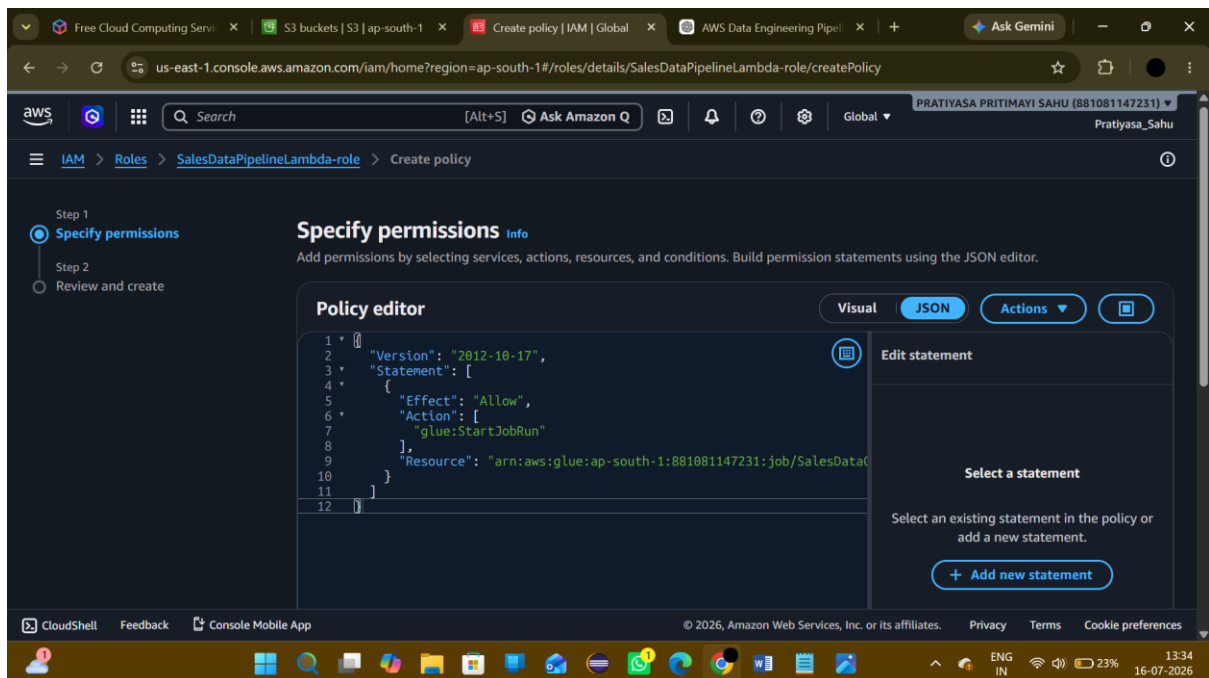
Created cleaned/ folder in processed-zone bucket



Lambda function iam role for permission and policies



Permission to start glue job



Data Pipeline lambda role

The screenshot shows the AWS IAM console's Policy editor for the role 'SalesDataPipelineLambda-role'. The interface is in the 'JSON' tab, displaying a policy document with two statements: 'ReadLandingBucket' and 'WriteRawBucket'. The 'ReadLandingBucket' statement allows 's3:GetObject' and 's3:ListBucket' actions on resources in the 'pratiyasa-landing-zone'. The 'WriteRawBucket' statement allows 's3:PutObject' actions on resources in the 'pratiyasa-raw-zone'.

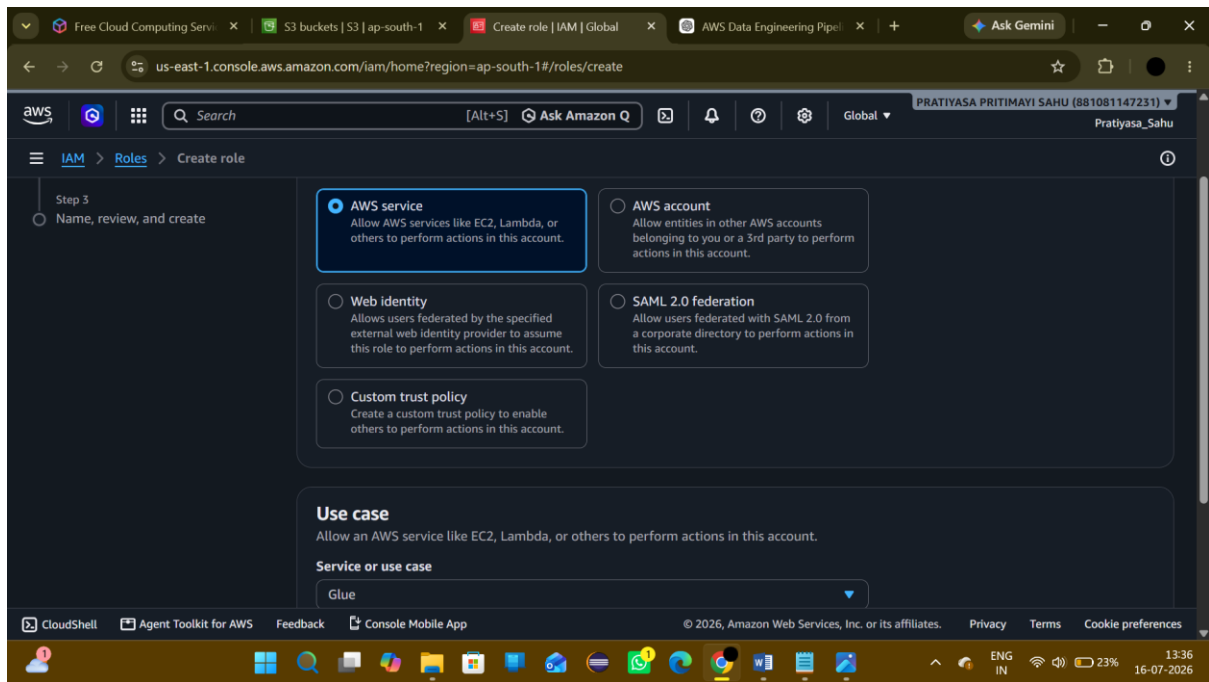
```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Sid": "ReadLandingBucket",
6       "Effect": "Allow",
7       "Action": [
8         "s3:GetObject",
9         "s3:ListBucket"
10      ],
11      "Resource": [
12        "arn:aws:s3:::pratiyasa-landing-zone",
13        "arn:aws:s3:::pratiyasa-landing-zone/*"
14      ]
15    },
16    {
17      "Sid": "WriteRawBucket",
18      "Effect": "Allow",
19      "Action": [
20        "s3:PutObject"
21      ],
22      "Resource": [
23        "arn:aws:s3:::pratiyasa-raw-zone/*"
24      ]
25    }
26  ]
27 }
```

All permission policies for lambda function

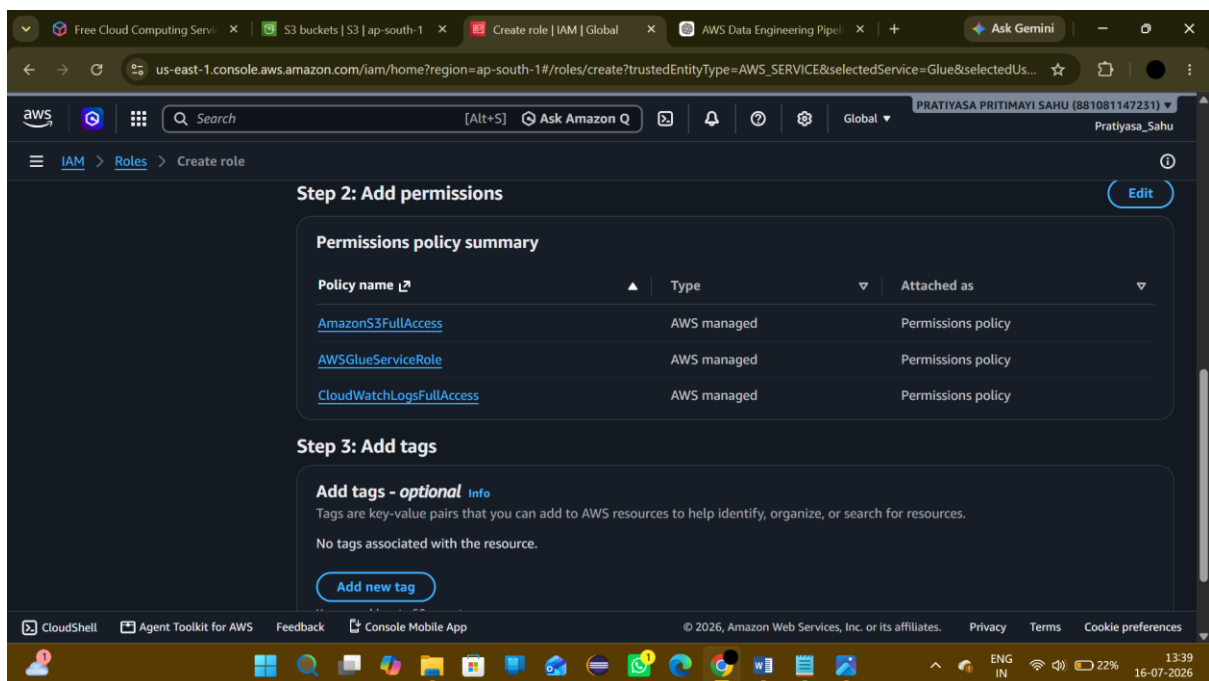
The screenshot shows the AWS IAM console's 'Permissions policies' page for the role 'SalesDataPipelineLambda-role'. It lists three policies: 'AWSLambdaBasicExecutionRole', 'gluejobaccess', and 'SalesDataPipelineLambda-rol...'. The 'Permissions boundary' is set to '(not set)'. There is a section for 'Generate policy based on CloudTrail events'.

Policy name	Type	Attached entities
AWSLambdaBasicExecutionRole	AWS managed	1
gluejobaccess	Customer inline	0
SalesDataPipelineLambda-rol...	Customer inline	0

Iam role permission for glue jobs



All permissions for glue jobs



All iam role permission for glue job crawler

The screenshot shows the AWS IAM console 'Create role' wizard. The role is being created for the `Glue` service. The permissions policy summary shows two attached policies: `AmazonS3FullAccess` and `AWSGlueServiceRole`, both of which are AWS managed. The 'Add tags' step is optional and currently has no tags associated with the resource.

```
9      "Action": "sts:AssumeRole"
10    }
11  ]
12 }
```

Step 2: Add permissions

Permissions policy summary

Policy name	Type	Attached as
AmazonS3FullAccess	AWS managed	Permissions policy
AWSGlueServiceRole	AWS managed	Permissions policy

Step 3: Add tags

Add tags - optional [Info](#)

Tags are key-value pairs that you can add to AWS resources to help identify, organize, or search for resources.

No tags associated with the resource.

All iam roles for required aws services

The screenshot shows the AWS IAM console 'Roles' page. A green notification banner at the top states 'Role DataPipelineCrawlerRole created.' The 'Roles (3/6)' list shows three roles created for the Glue service and one for the Lambda service. The 'DataPipelineCrawlerRole' is selected.

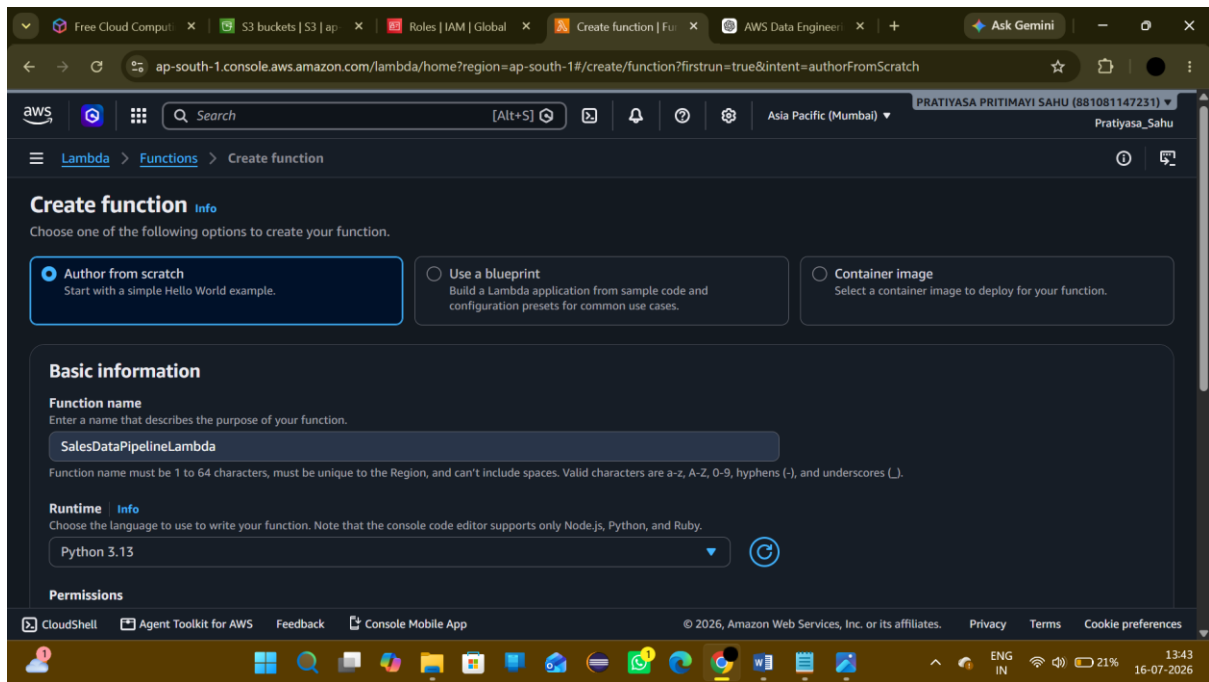
Role DataPipelineCrawlerRole created. [View role](#)

Roles (3/6) [Info](#) [Delete](#) [Create role](#)

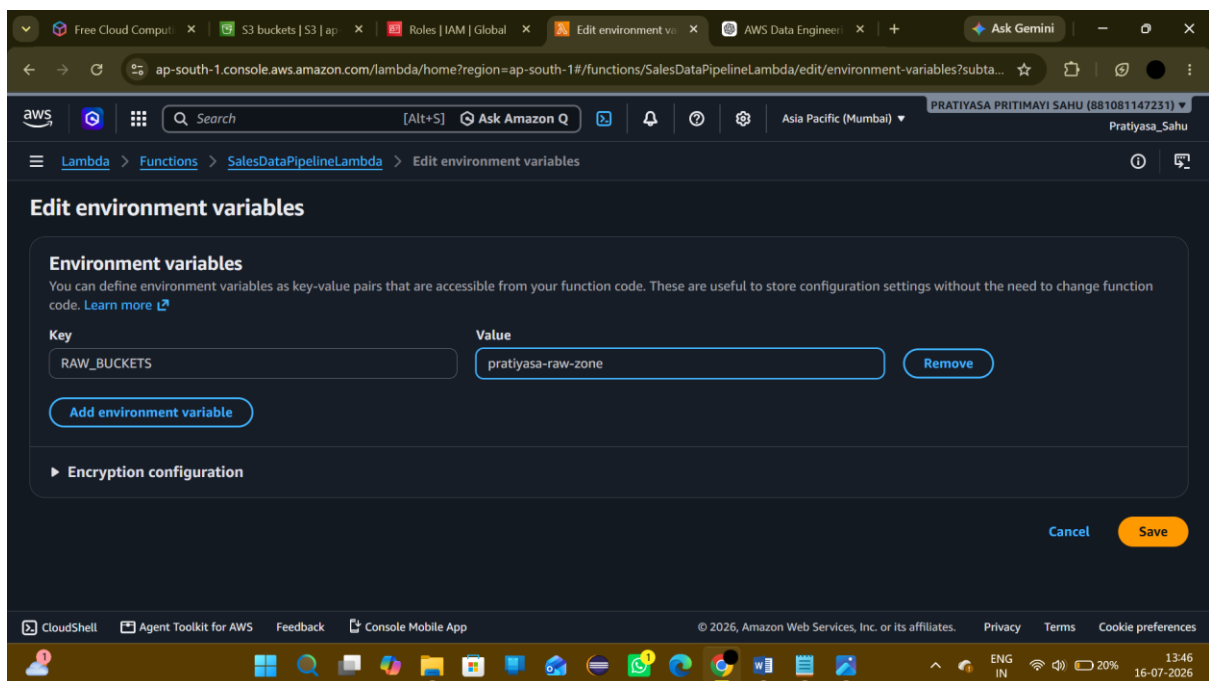
An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Role name	Trusted entities	Last activity
☐ AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service	-
☒ DataPipelineCrawlerRole	AWS Service: glue	-
☒ DataPipelineGlueRole	AWS Service: glue	-
☒ SalesDataPipelineLambda-role	AWS Service: lambda	-

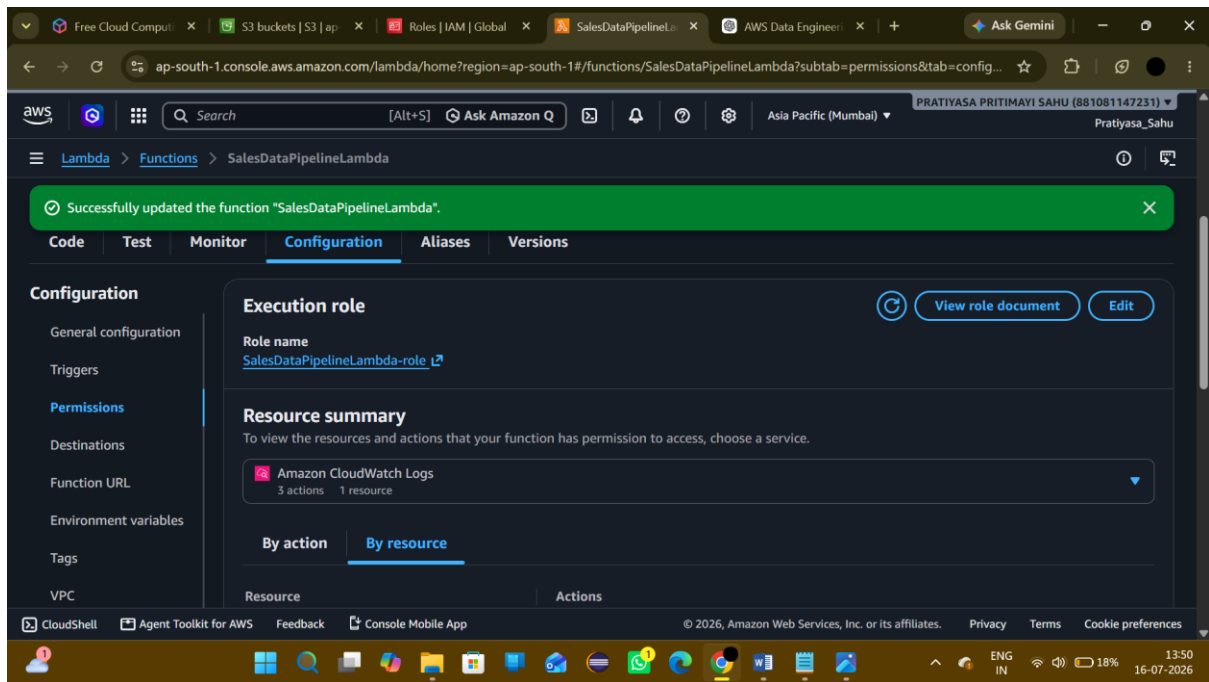
Created a Lambda function to copy content from landing-zone to raw-zone and to trigger the DataCleaningGlue job



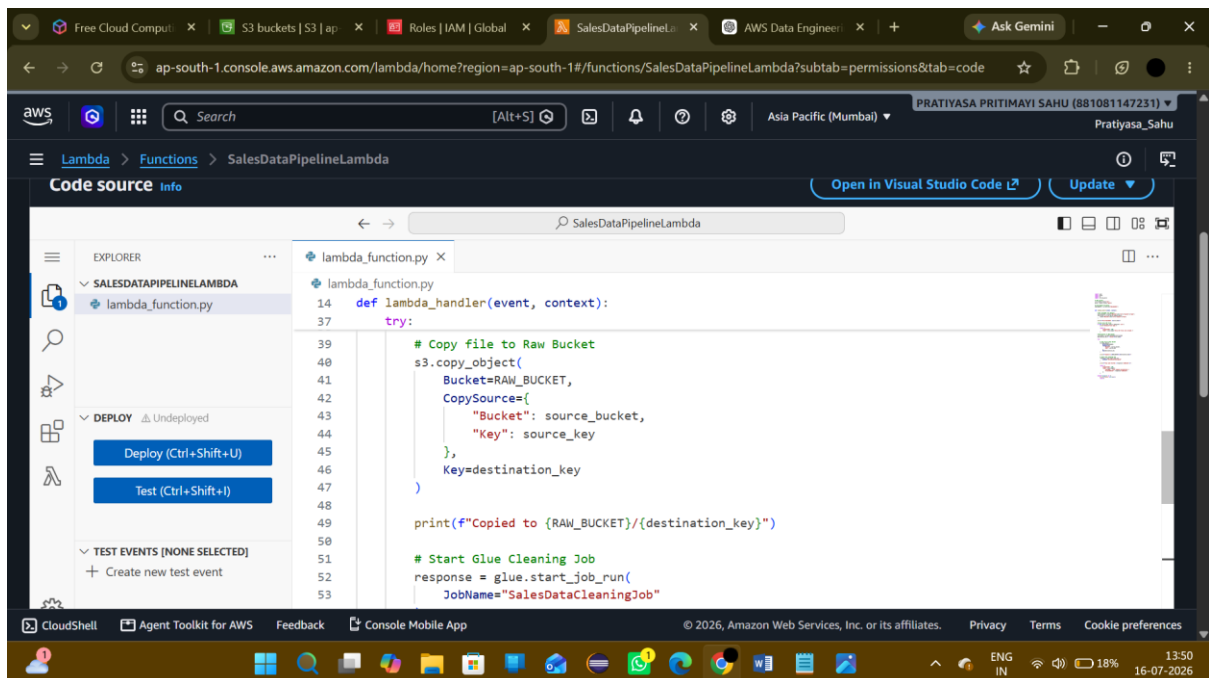
Environment Variable Added for lambda function



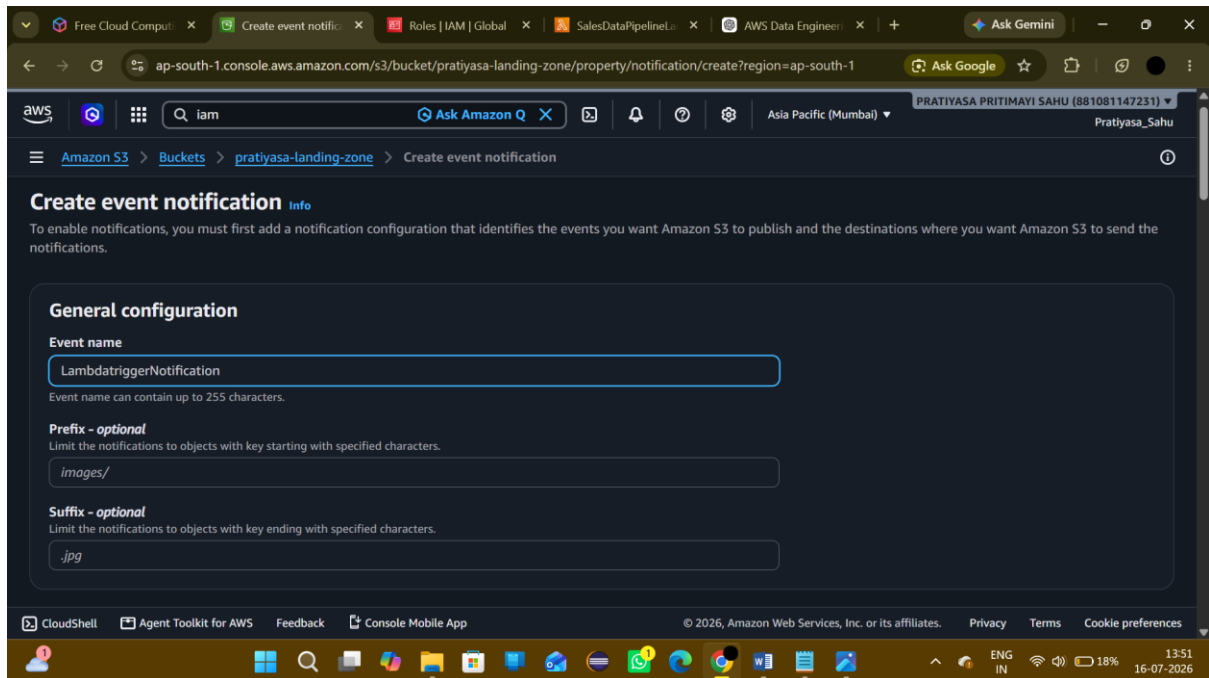
Attaching the DataPipelineLambda-role to the created lambda function



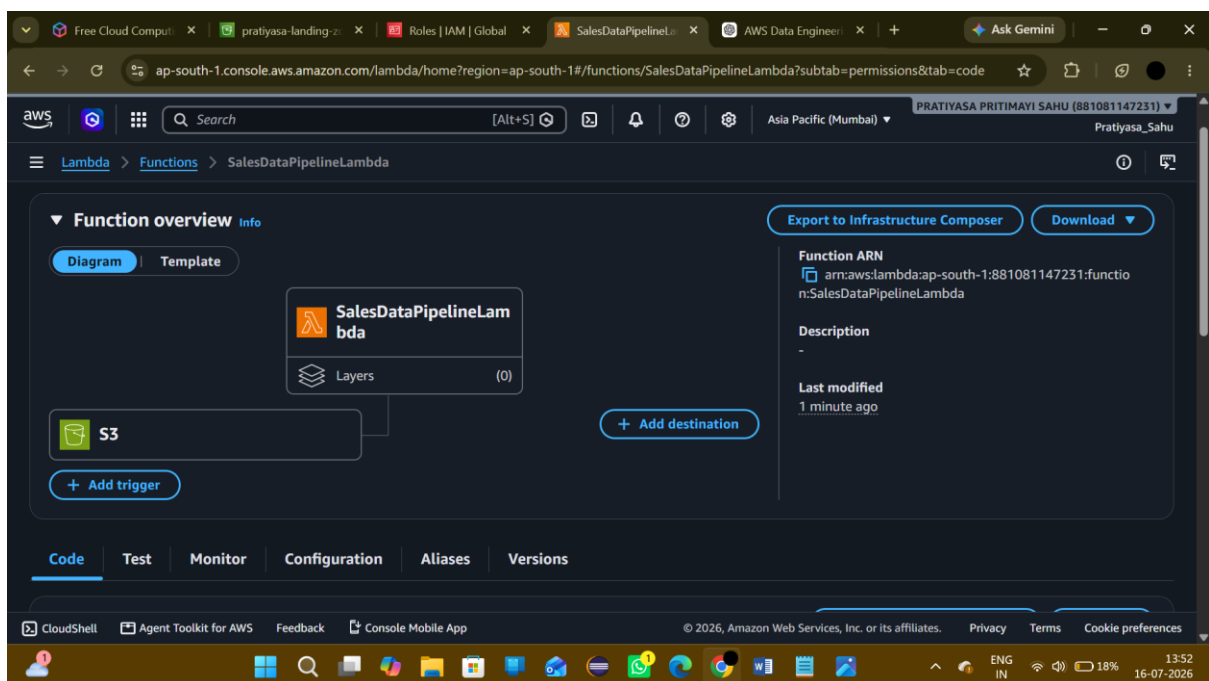
Lambda Script to copy the content and start the glue job



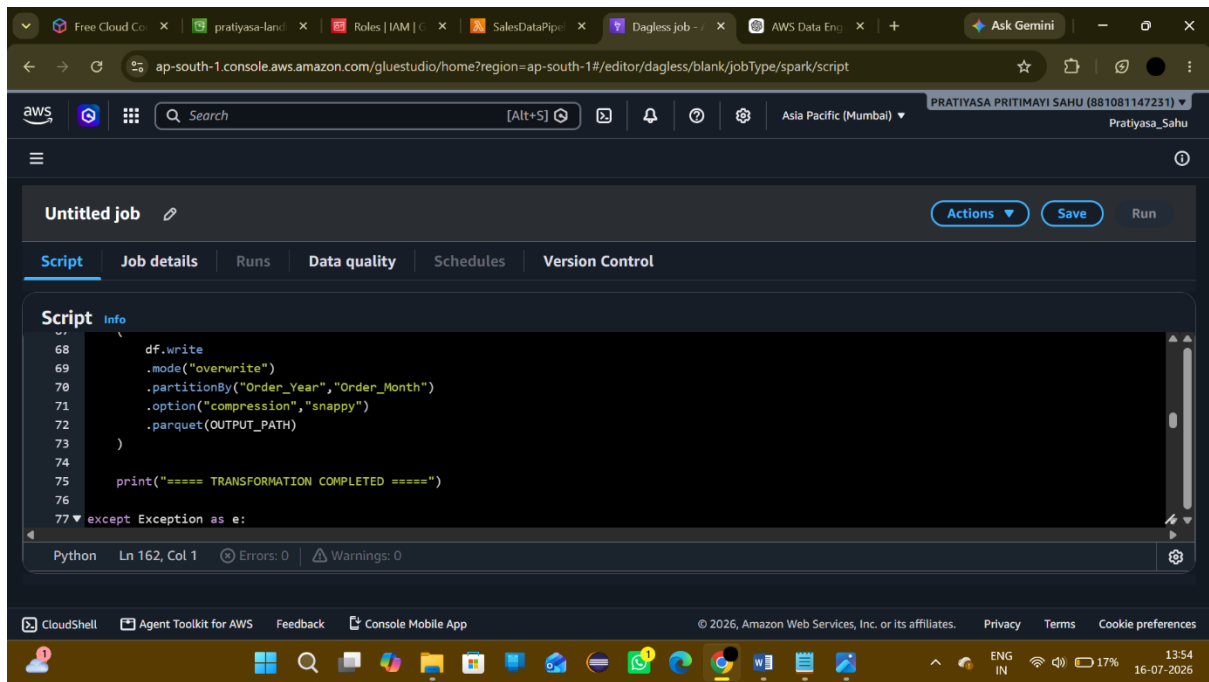
Creating event notification for landing-zone S3 bucket to trigger lambda function



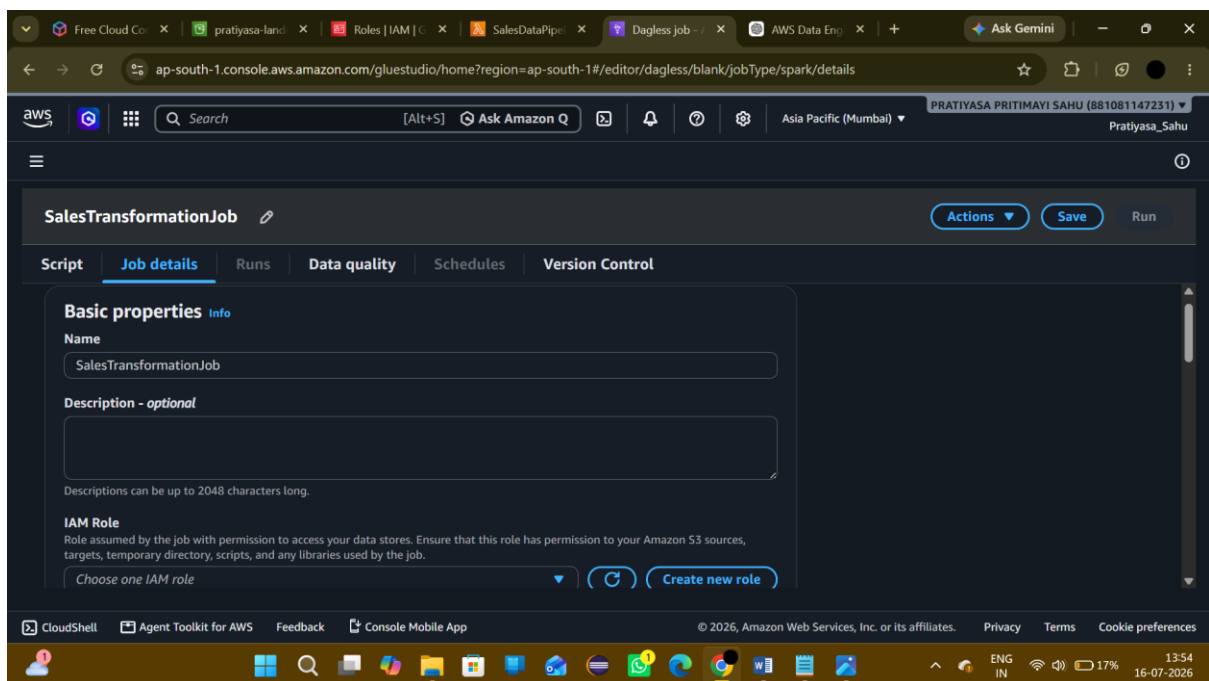
Lambda Function Overview



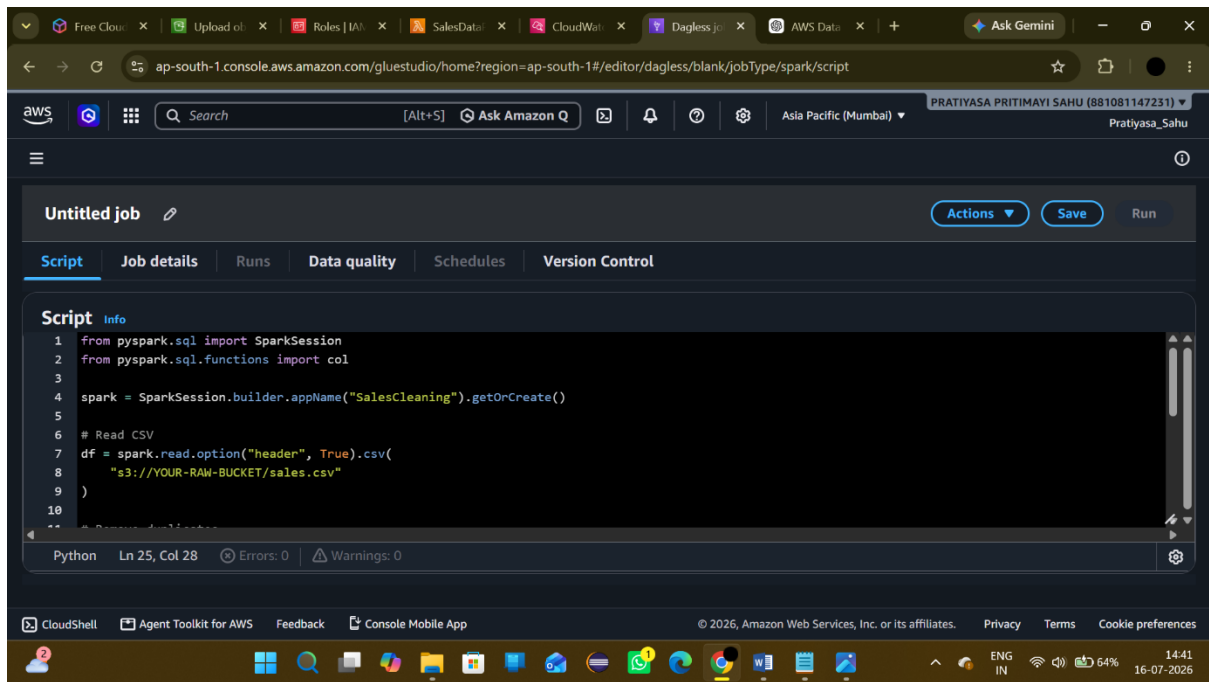
Python Script for Glue Transformation job



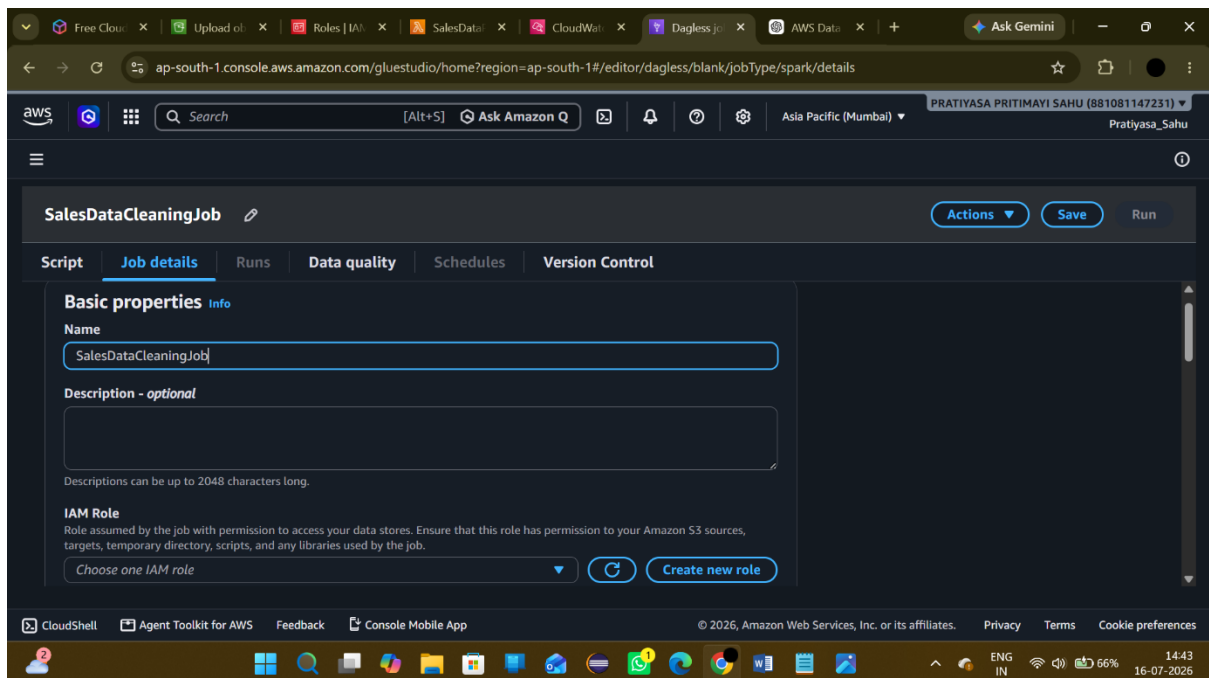
Details of Glue Transformation job



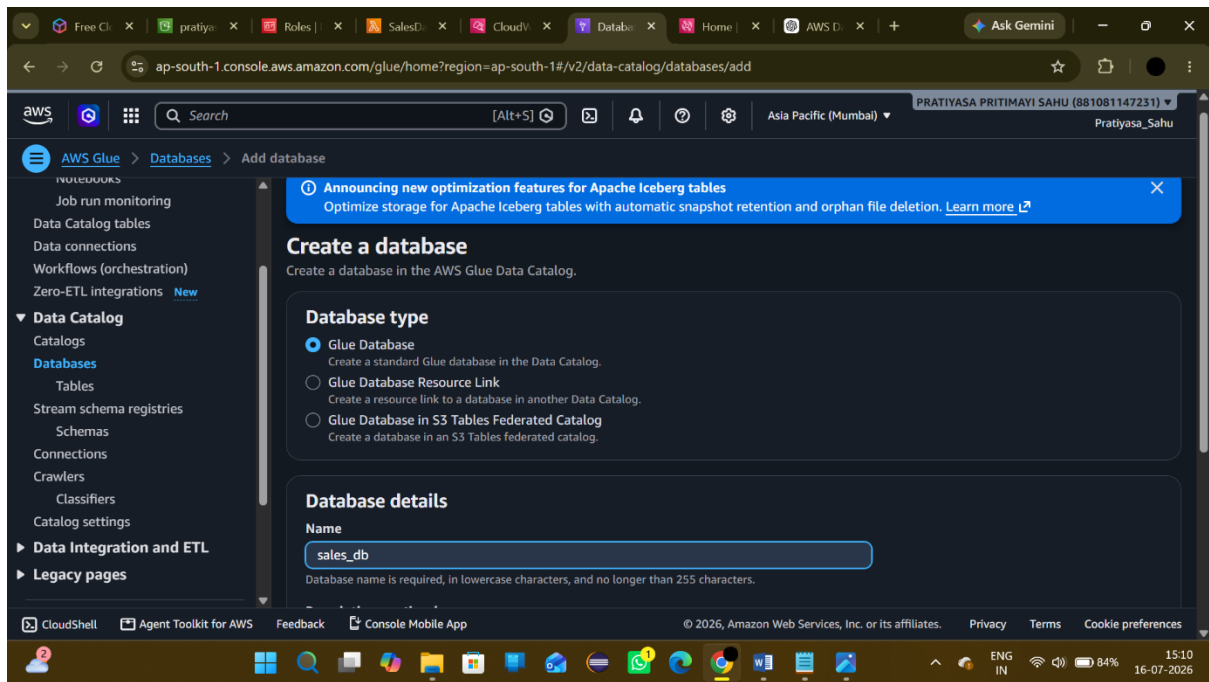
Python Script for Glue data cleaning job



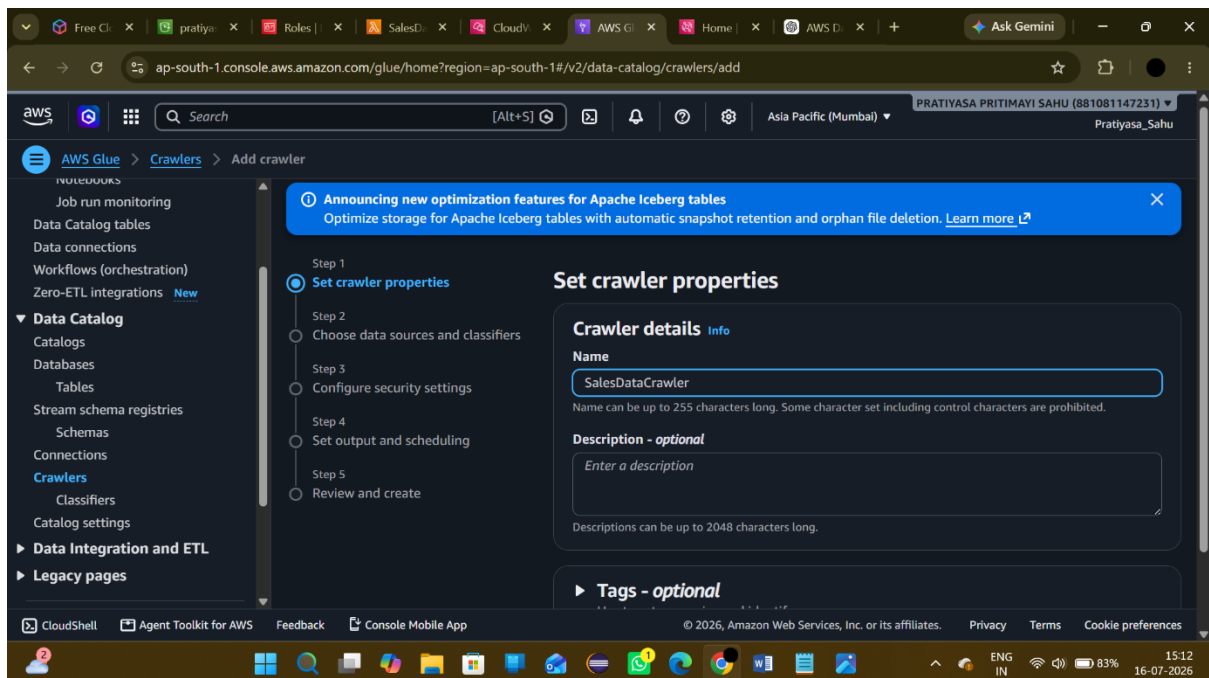
Details of Glue Data Cleaning job



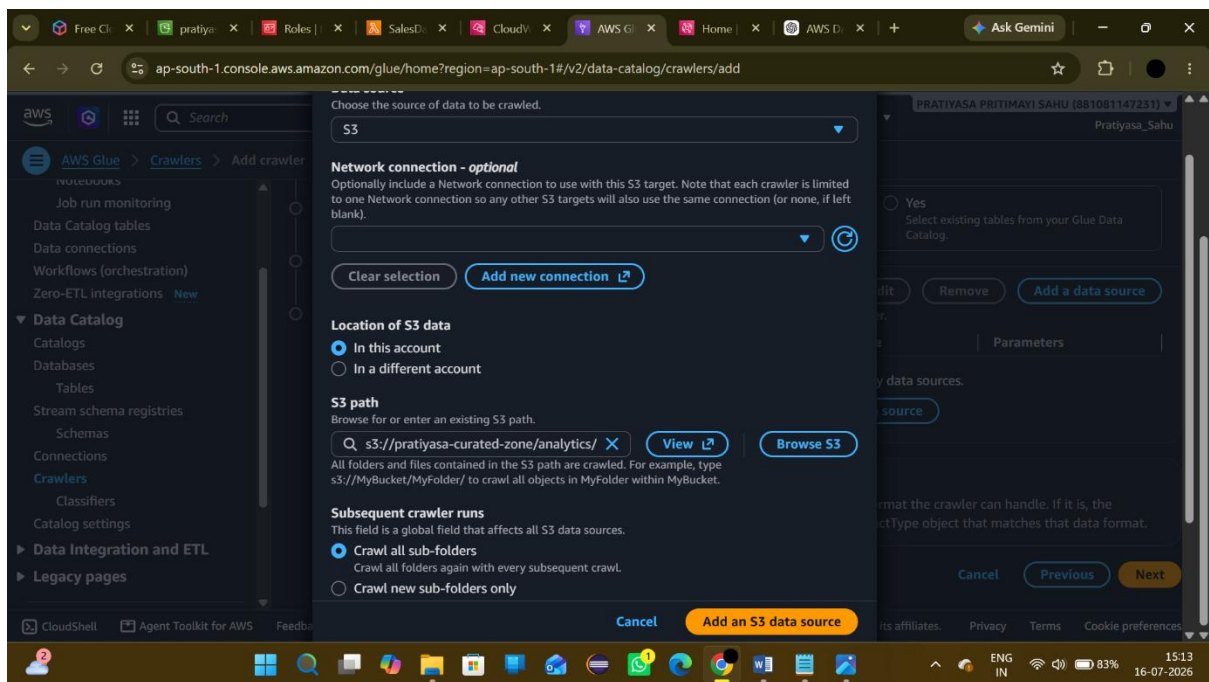
Creating Glue Database



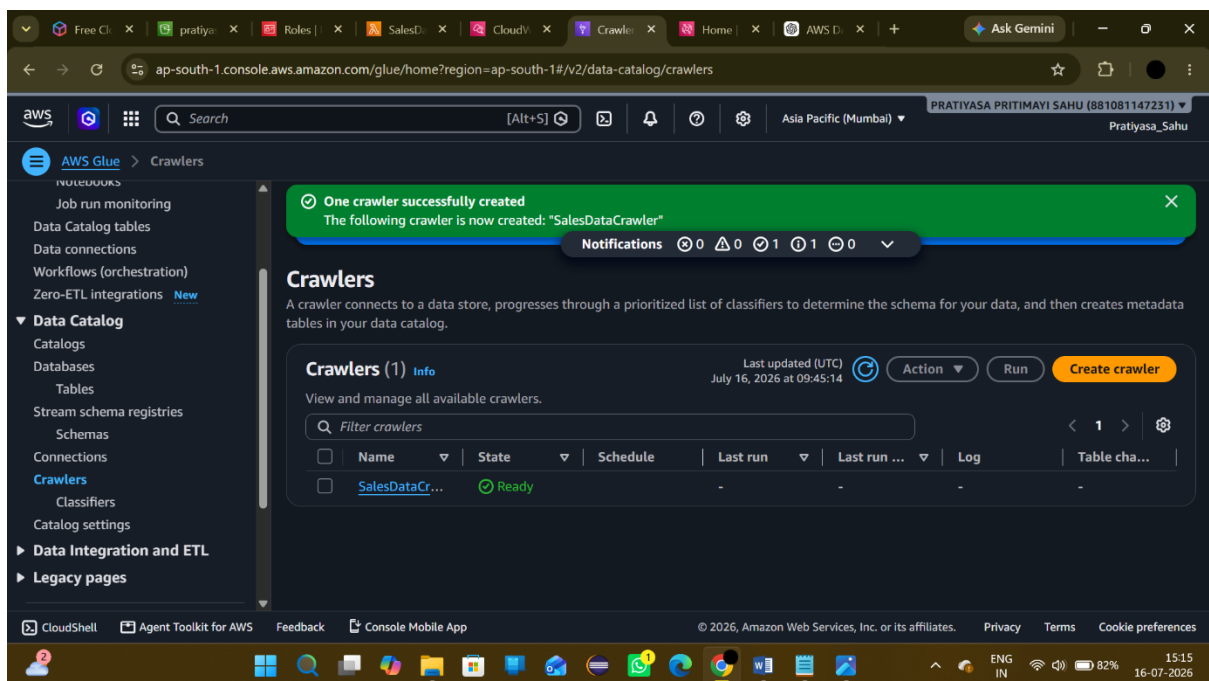
Creating Glue Data Crawler



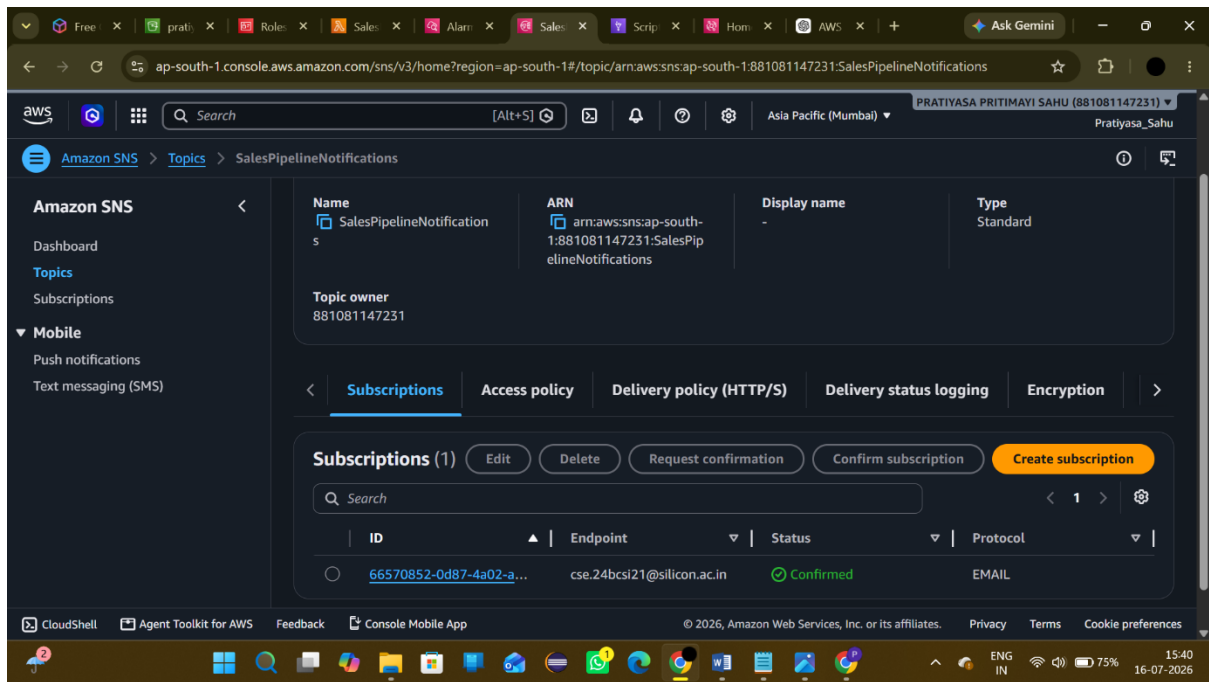
Defining the path for curated zone S3 bucket to run crawler



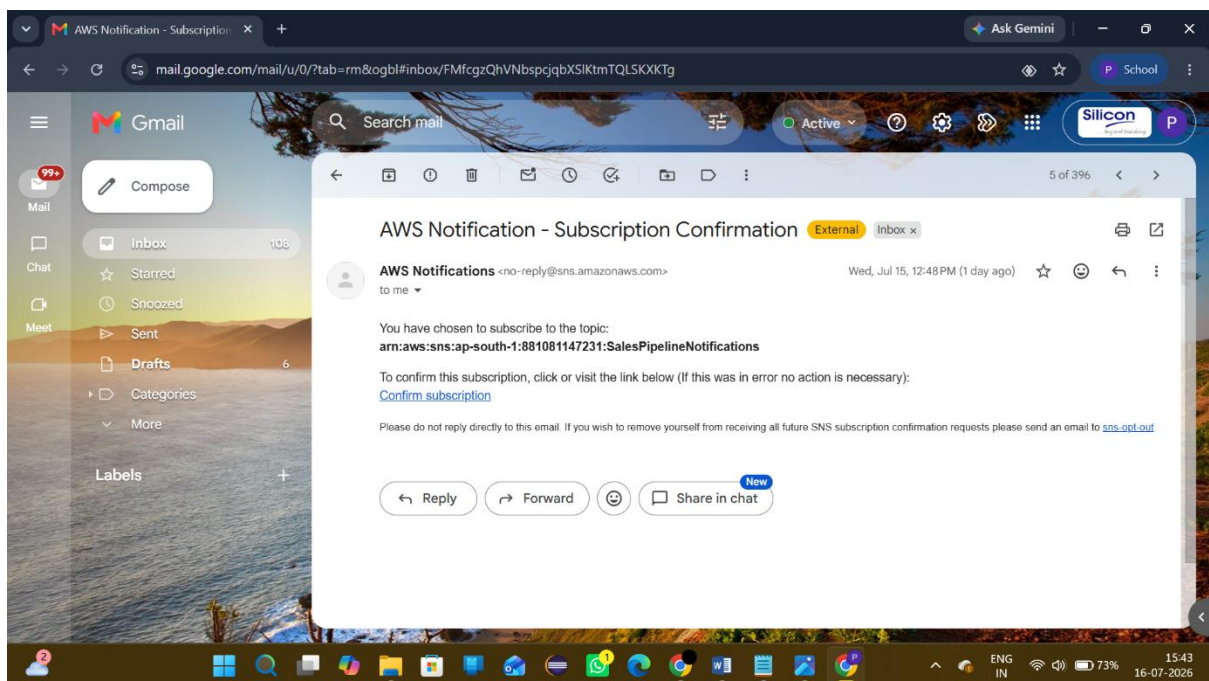
Crawler job created



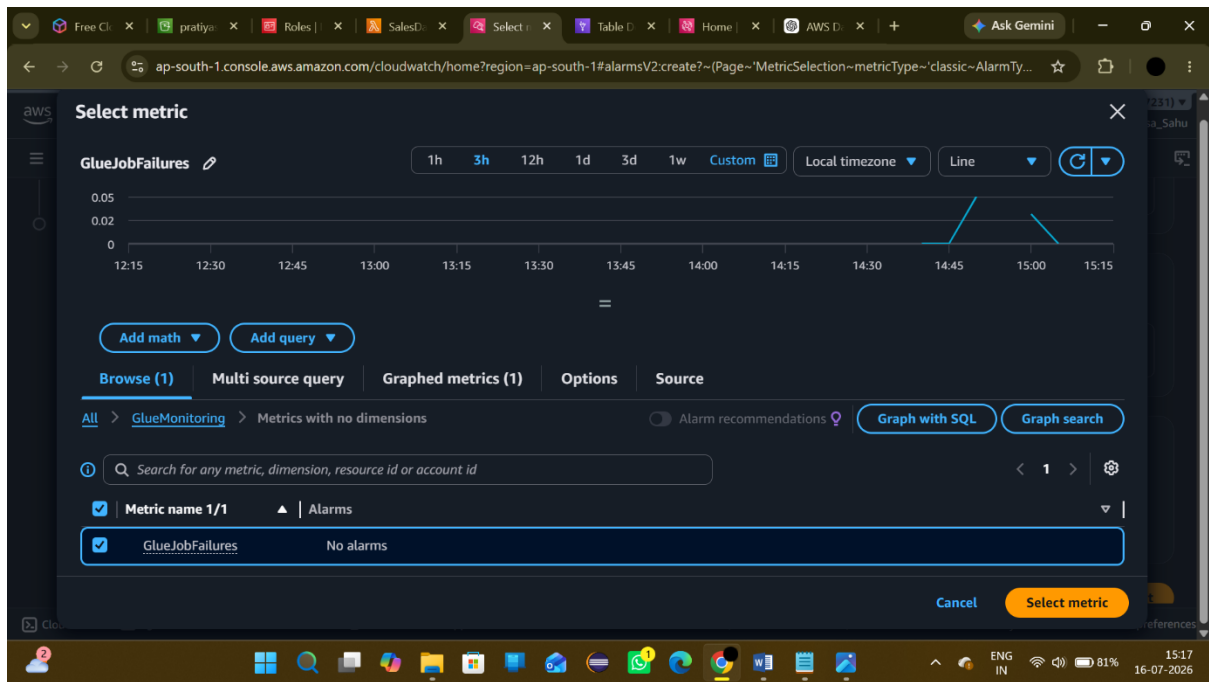
Configured SNS for notifications



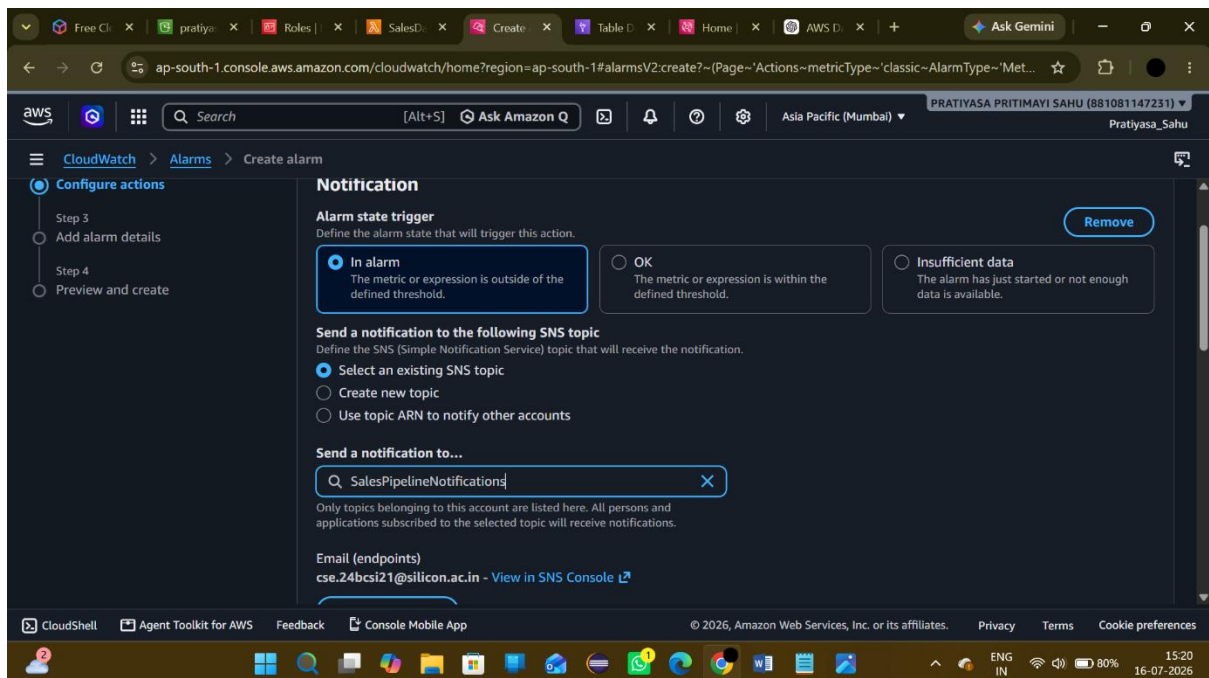
Confirming Subscription for email sns notification



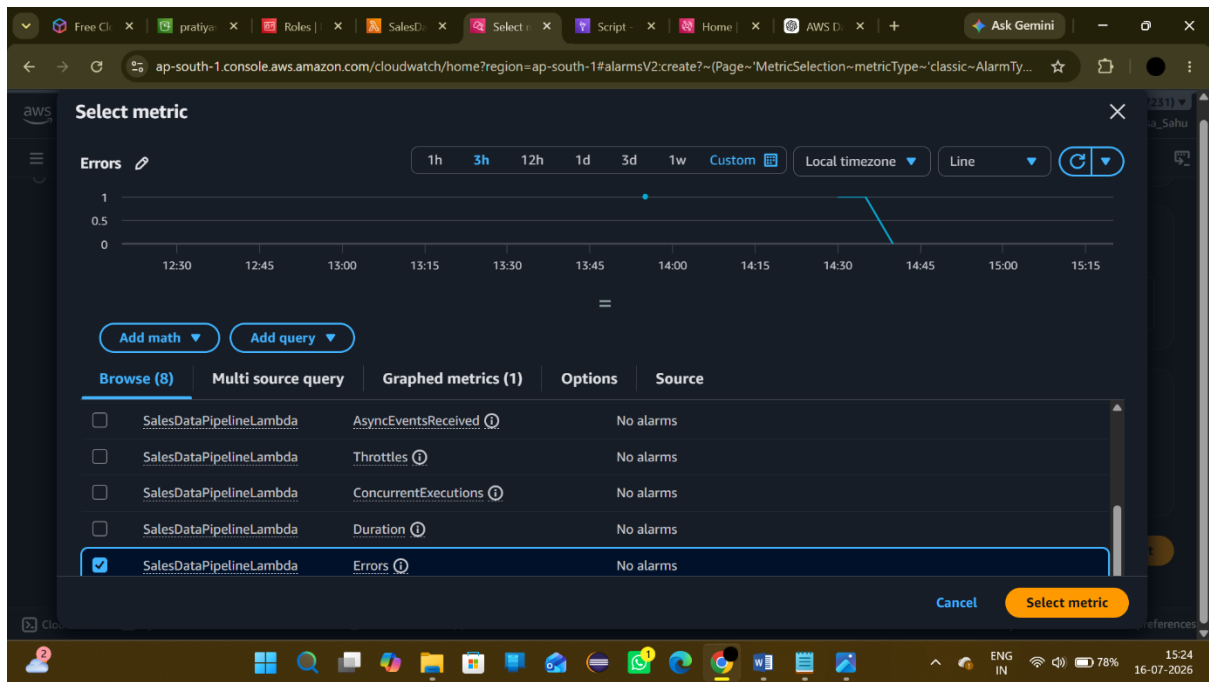
Selecting metric for glue job failure to create alarm



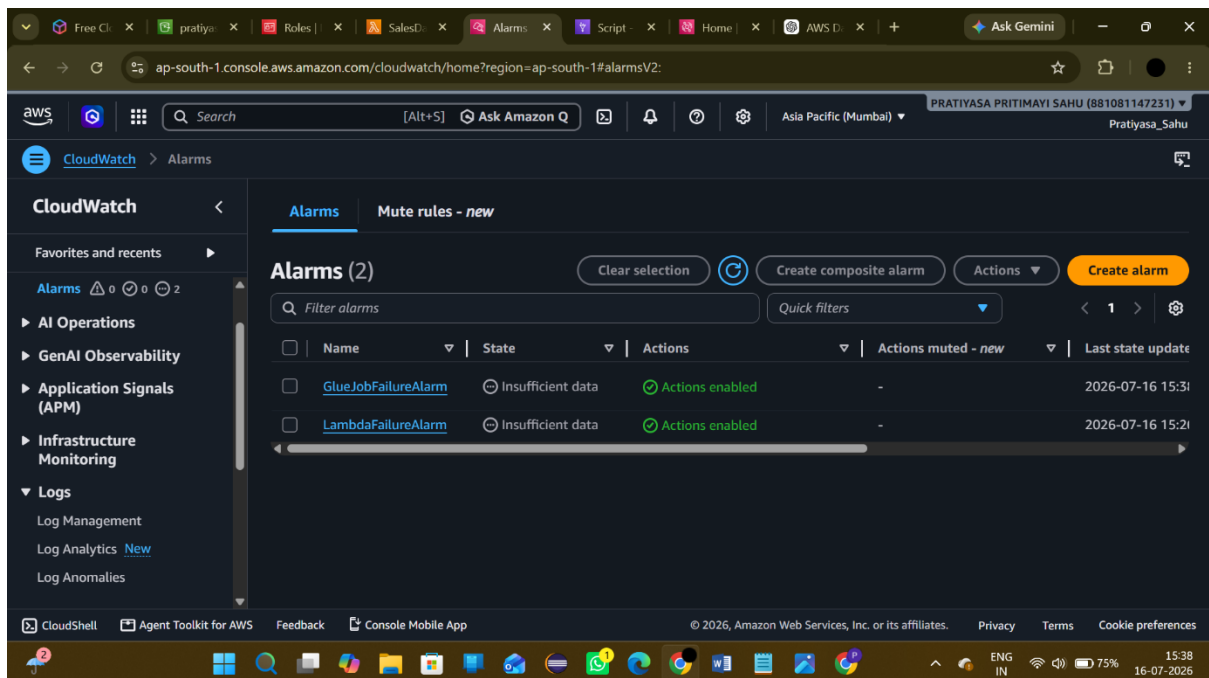
Selecting the endpoint of notification (email sns endpoint)



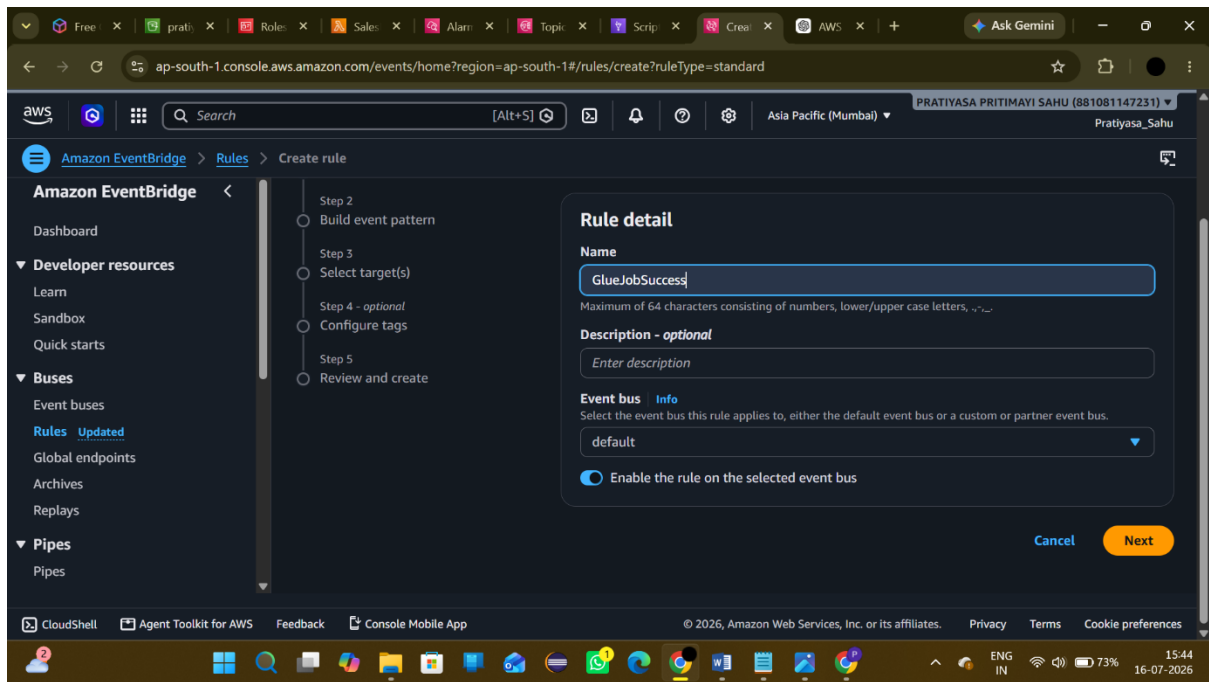
Selecting the metric for lambda function failure to create alarm



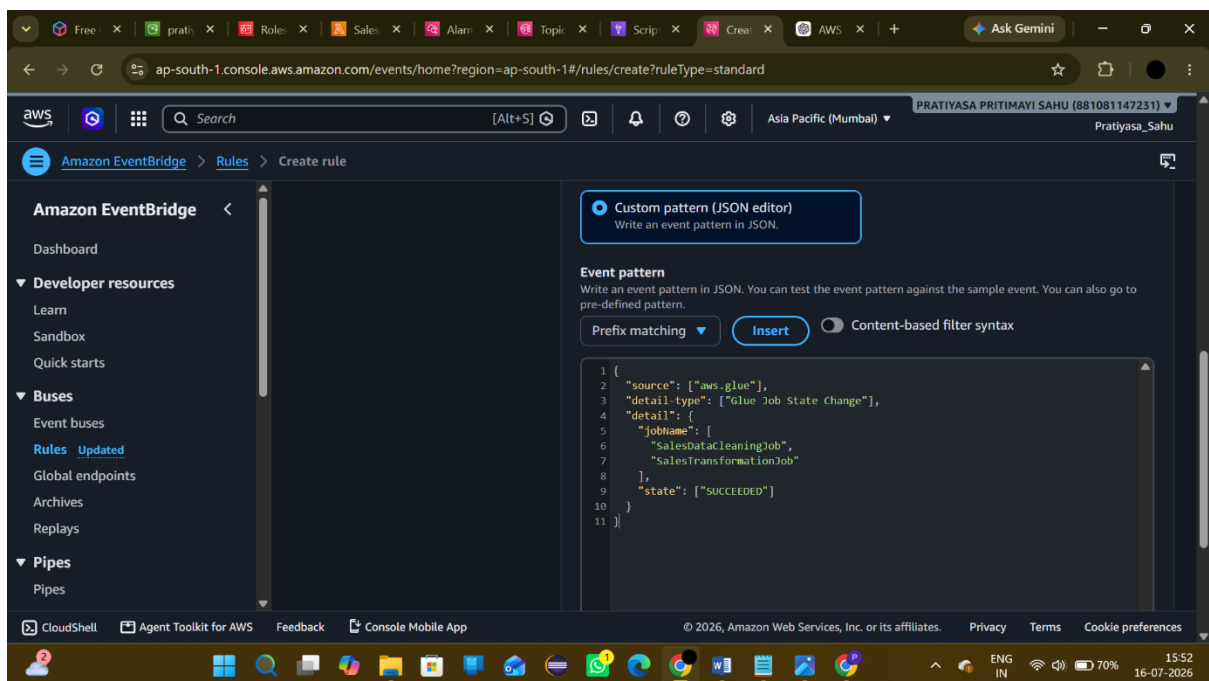
2 alarms configured for failure



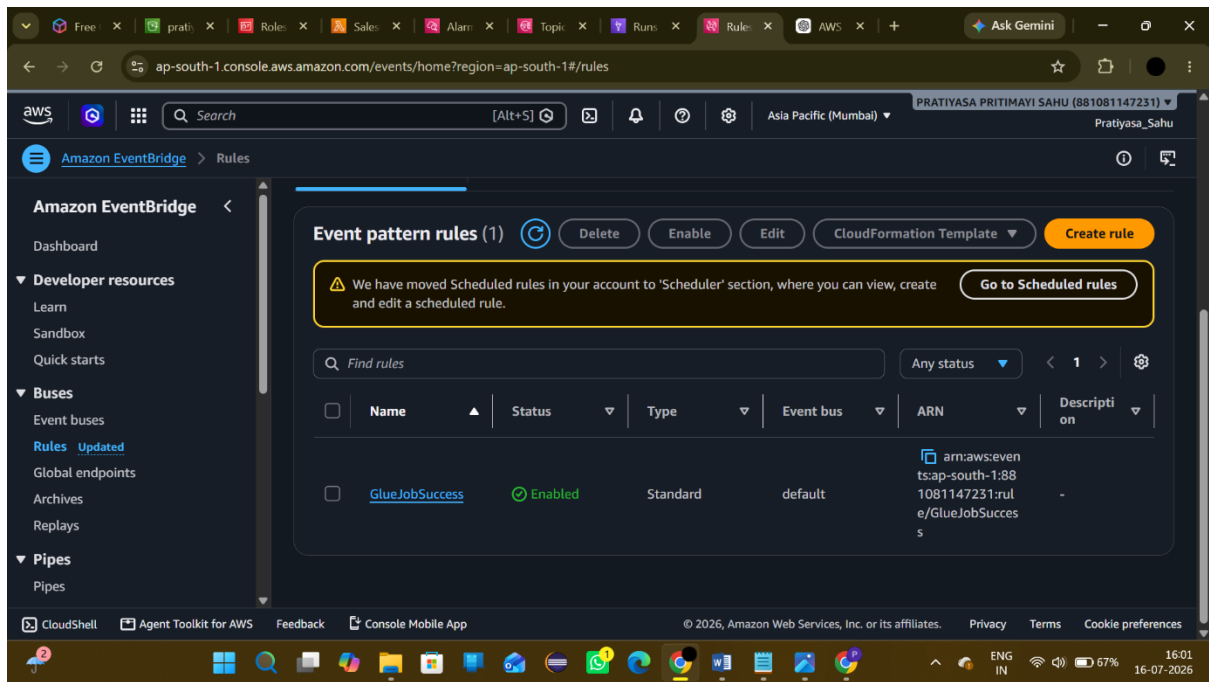
Glue jobs success Eventbridge rule



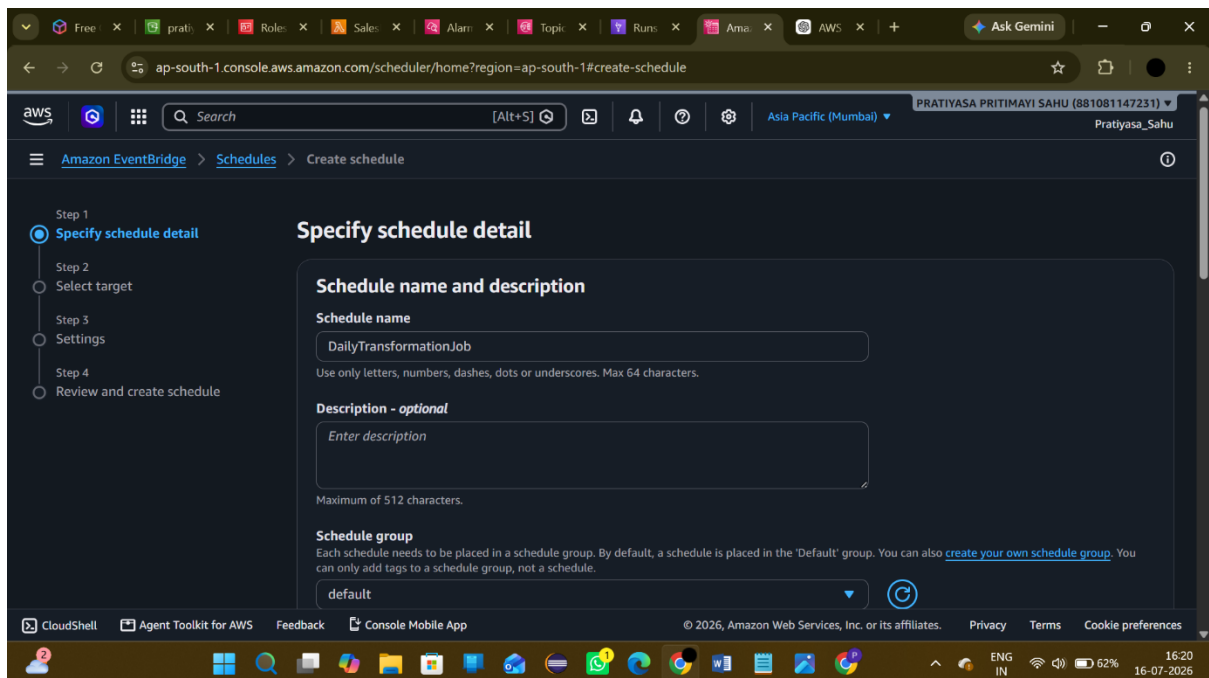
Custom json pattern for selected glue jobs



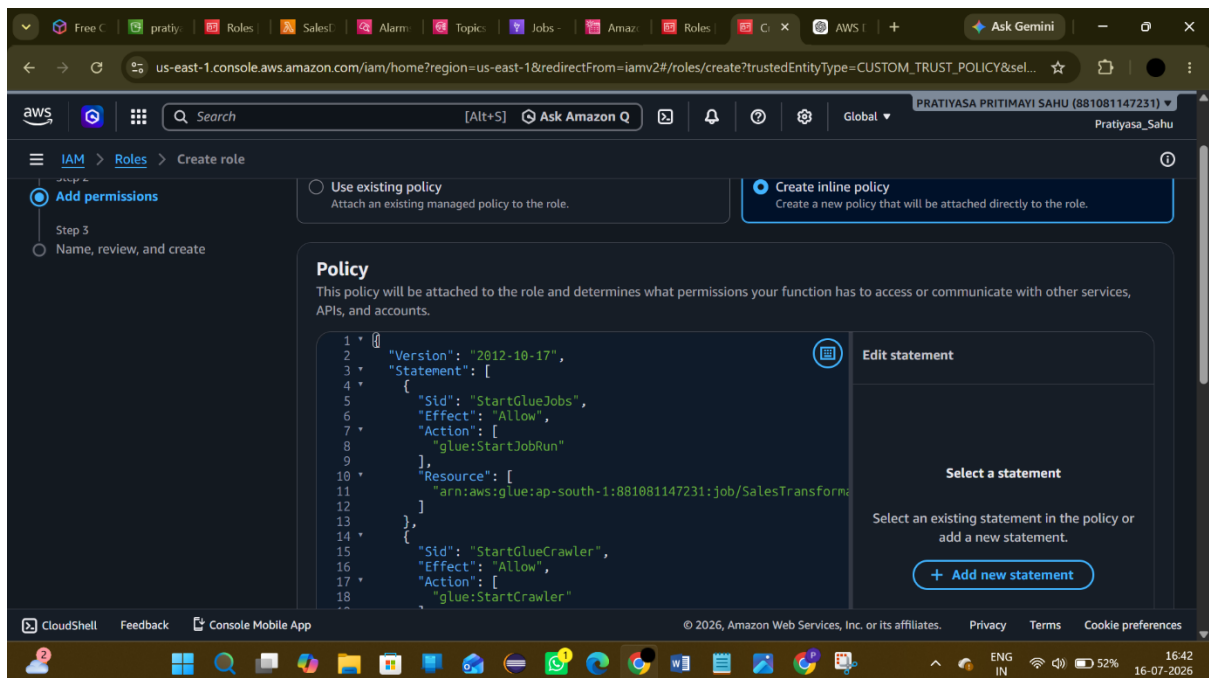
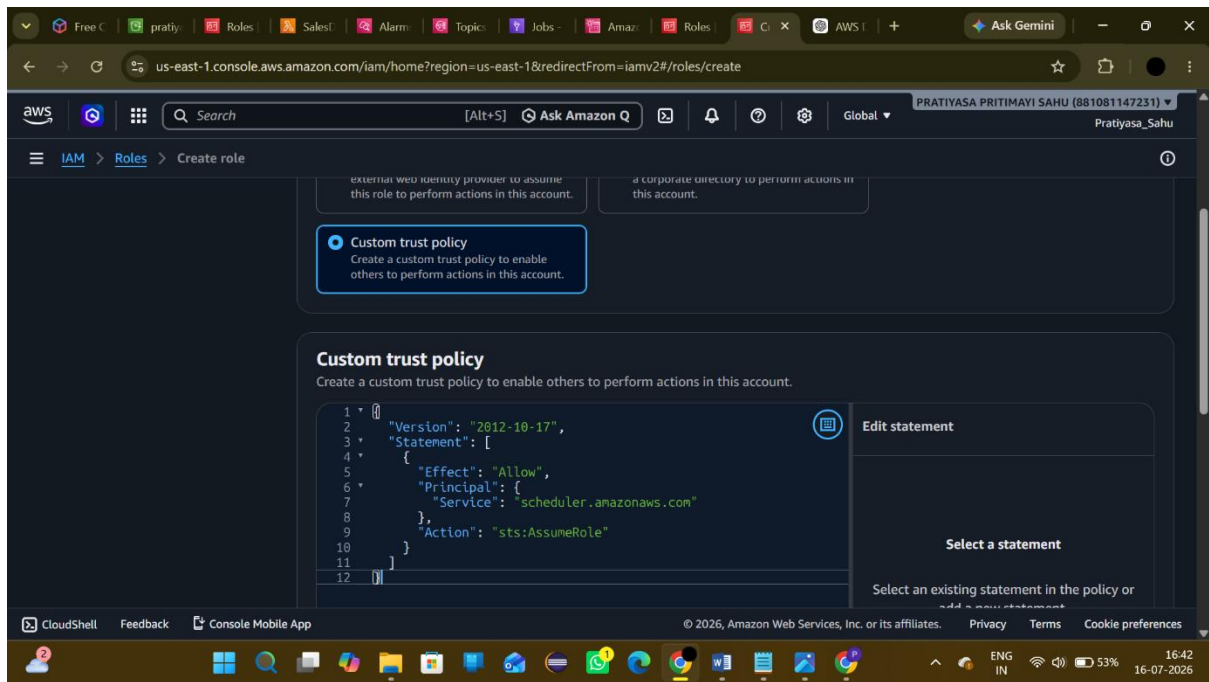
Gluejobsuccess eventrule created



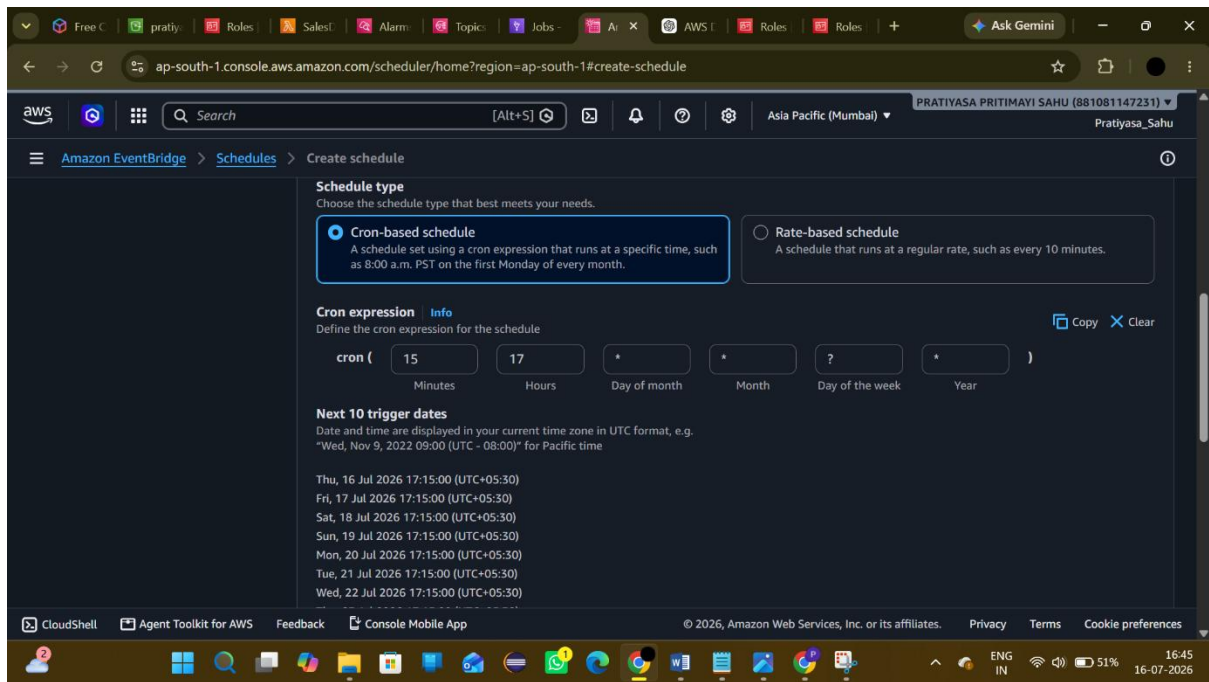
Scheduling the transformation glue job through eventbridge scheduler



Custom trust policy iam role to allow scheduler to trigger the glue job



Scheduled to trigger at 5:15 PM IST everyday



ap-south-1.console.aws.amazon.com/scheduler/home?region=ap-south-1#create-schedule

Amazon EventBridge > Schedules > Create schedule

Schedule type
Choose the schedule type that best meets your needs.

☒ **Cron-based schedule**
A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.

☐ **Rate-based schedule**
A schedule that runs at a regular rate, such as every 10 minutes.

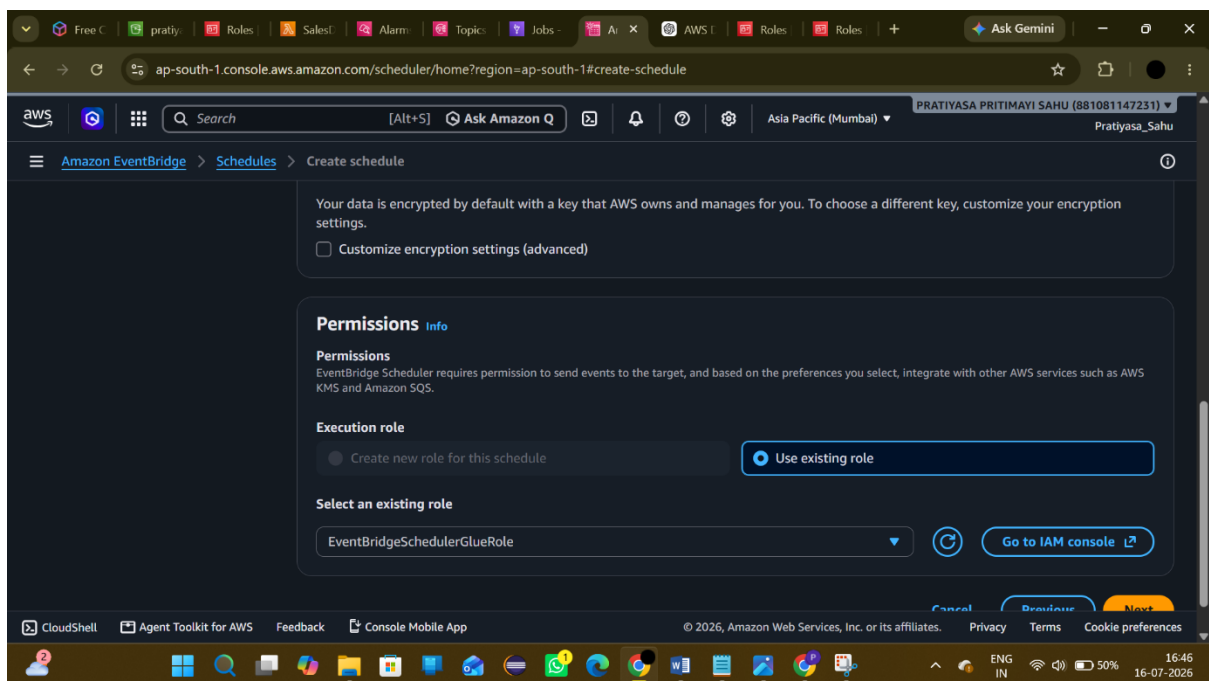
Cron expression [Info](#)
Define the cron expression for the schedule

cron (15 17 * * ? *)
Minutes Hours Day of month Month Day of the week Year

Next 10 trigger dates
Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time

Thu, 16 Jul 2026 17:15:00 (UTC+05:30)
Fri, 17 Jul 2026 17:15:00 (UTC+05:30)
Sat, 18 Jul 2026 17:15:00 (UTC+05:30)
Sun, 19 Jul 2026 17:15:00 (UTC+05:30)
Mon, 20 Jul 2026 17:15:00 (UTC+05:30)
Tue, 21 Jul 2026 17:15:00 (UTC+05:30)
Wed, 22 Jul 2026 17:15:00 (UTC+05:30)

Selecting the created iam role policy



ap-south-1.console.aws.amazon.com/scheduler/home?region=ap-south-1#create-schedule

Amazon EventBridge > Schedules > Create schedule

Your data is encrypted by default with a key that AWS owns and manages for you. To choose a different key, customize your encryption settings.

☐ Customize encryption settings (advanced)

Permissions [Info](#)
EventBridge Scheduler requires permission to send events to the target, and based on the preferences you select, integrate with other AWS services such as AWS KMS and Amazon SQS.

Execution role

☐ Create new role for this schedule

☒ Use existing role

Select an existing role

EventBridgeSchedulerGlueRole

[Go to IAM console](#)

Scheduling the Data crawler to run after transformation job

ap-south-1.console.aws.amazon.com/scheduler/home?region=ap-south-1#create-schedule

Amazon EventBridge > Schedules > Create schedule

✓ Your schedule DailyTransformationJob is being created.

Step 2
Select target

Step 3
Settings

Step 4
Review and create schedule

Schedule name and description

Schedule name

DailySalesCrawler

Use only letters, numbers, dashes, dots or underscores. Max 64 characters.

Description - optional

Enter description

Maximum of 512 characters.

Schedule group

Each schedule needs to be placed in a schedule group. By default, a schedule is placed in the 'Default' group. You can also [create your own schedule group](#). You can only add tags to a schedule group, not a schedule.

default

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

16:47 16-07-2026

Scheduled at 5:20 PM IST everyday

ap-south-1.console.aws.amazon.com/scheduler/home?region=ap-south-1#create-schedule

Amazon EventBridge > Schedules > Create schedule

Schedule type

Choose the schedule type that best meets your needs.

☒ Cron-based schedule
A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.

☐ Rate-based schedule
A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression [Info](#)

Define the cron expression for the schedule

cron (20 17 * * ? *)

Minutes Hours Day of month Month Day of the week Year

Next 10 trigger dates

Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time

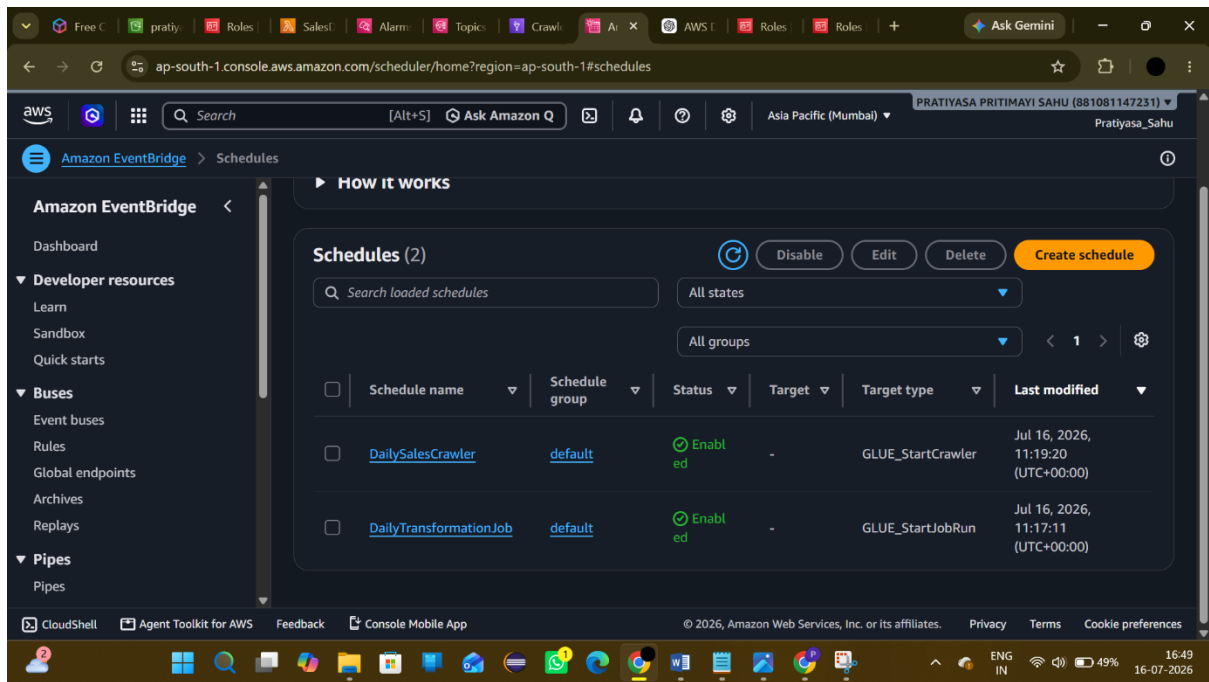
Thu, 16 Jul 2026 17:20:00 (UTC+05:30)
Fri, 17 Jul 2026 17:20:00 (UTC+05:30)
Sat, 18 Jul 2026 17:20:00 (UTC+05:30)
Sun, 19 Jul 2026 17:20:00 (UTC+05:30)
Mon, 20 Jul 2026 17:20:00 (UTC+05:30)
Tue, 21 Jul 2026 17:20:00 (UTC+05:30)
Wed, 22 Jul 2026 17:20:00 (UTC+05:30)

CloudShell Agent Toolkit for AWS Feedback Console Mobile App

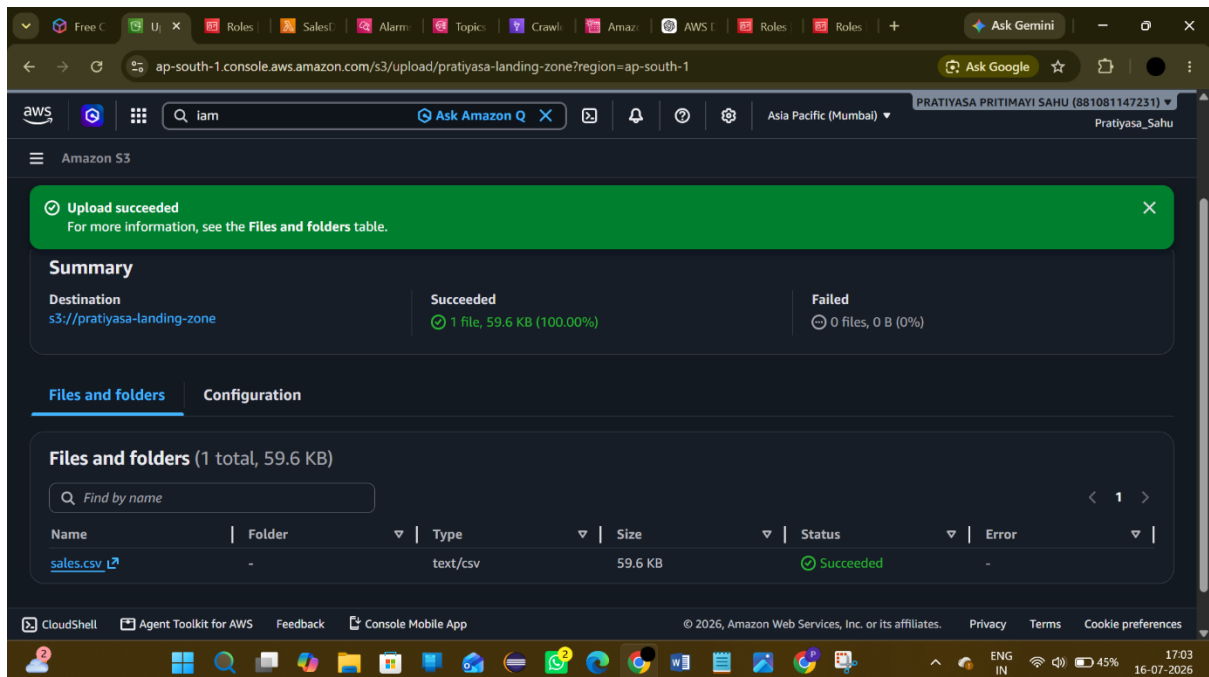
© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

16:47 16-07-2026

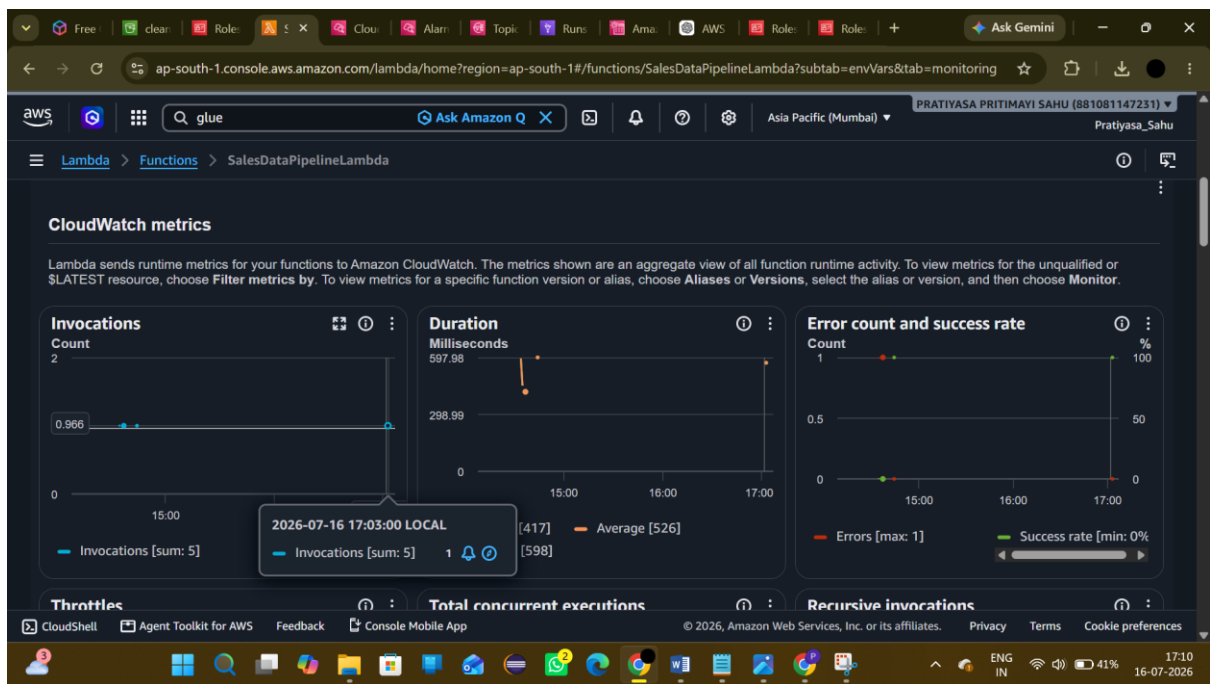
Both the Schedulers



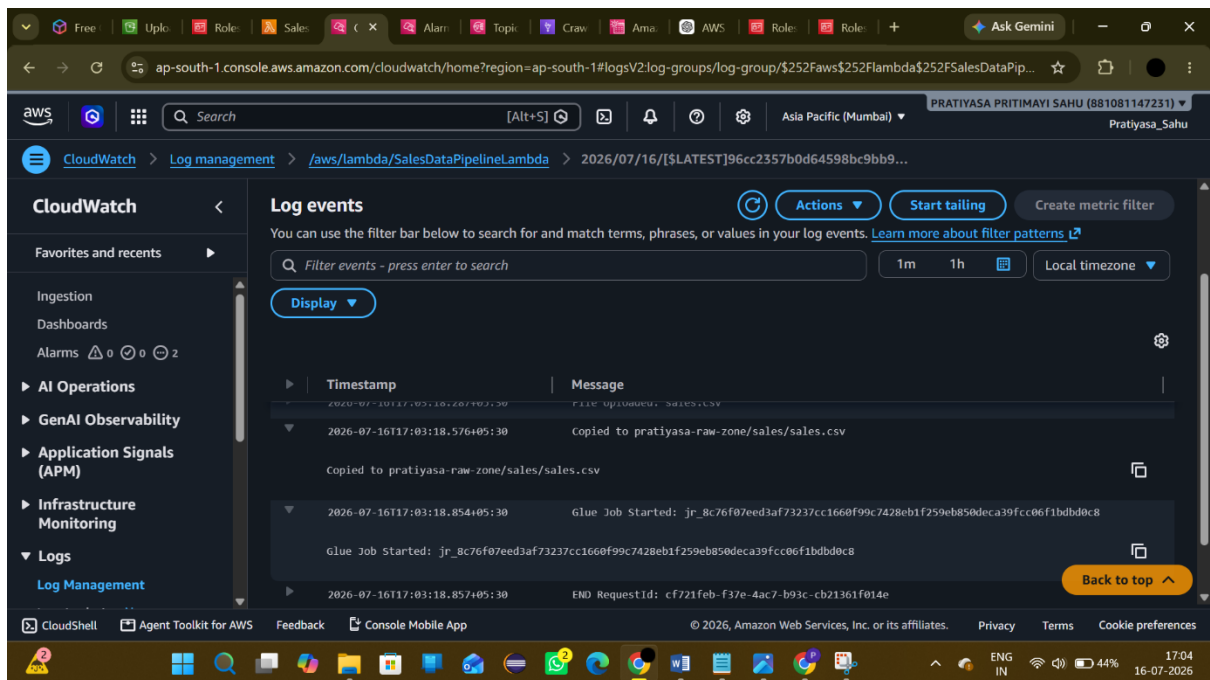
Starting the automation pipeline by uploading a csv file of sales table into landing zone S3 bucket



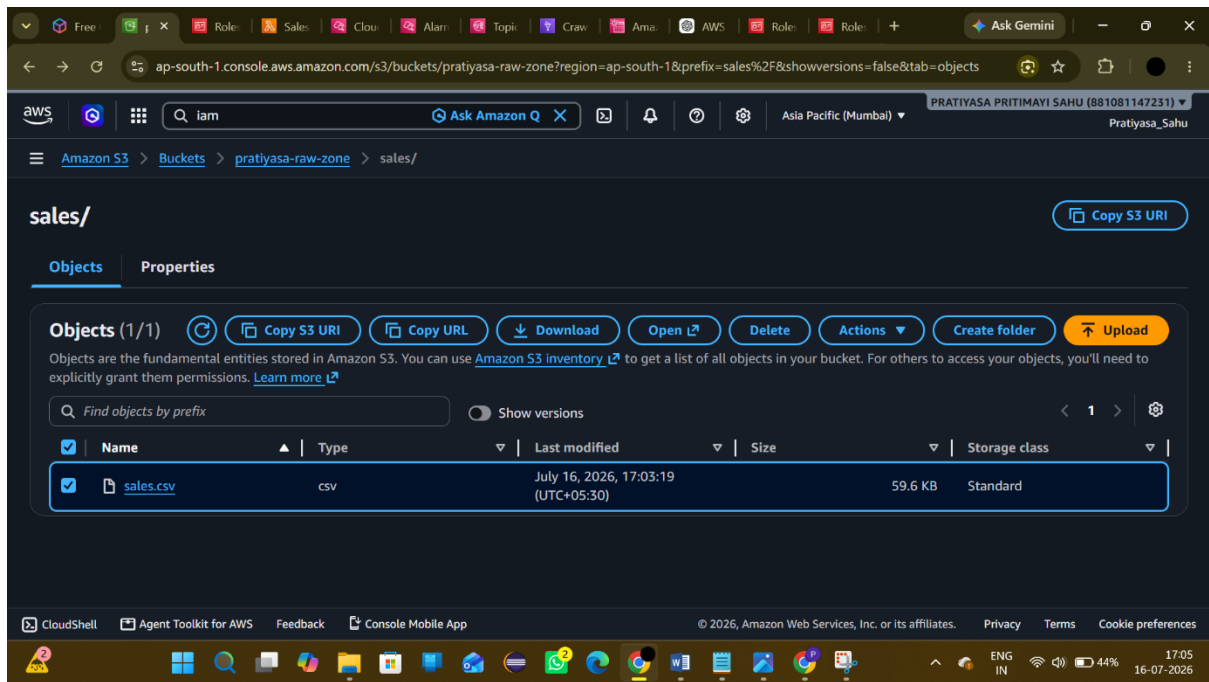
Lambda Function triggered



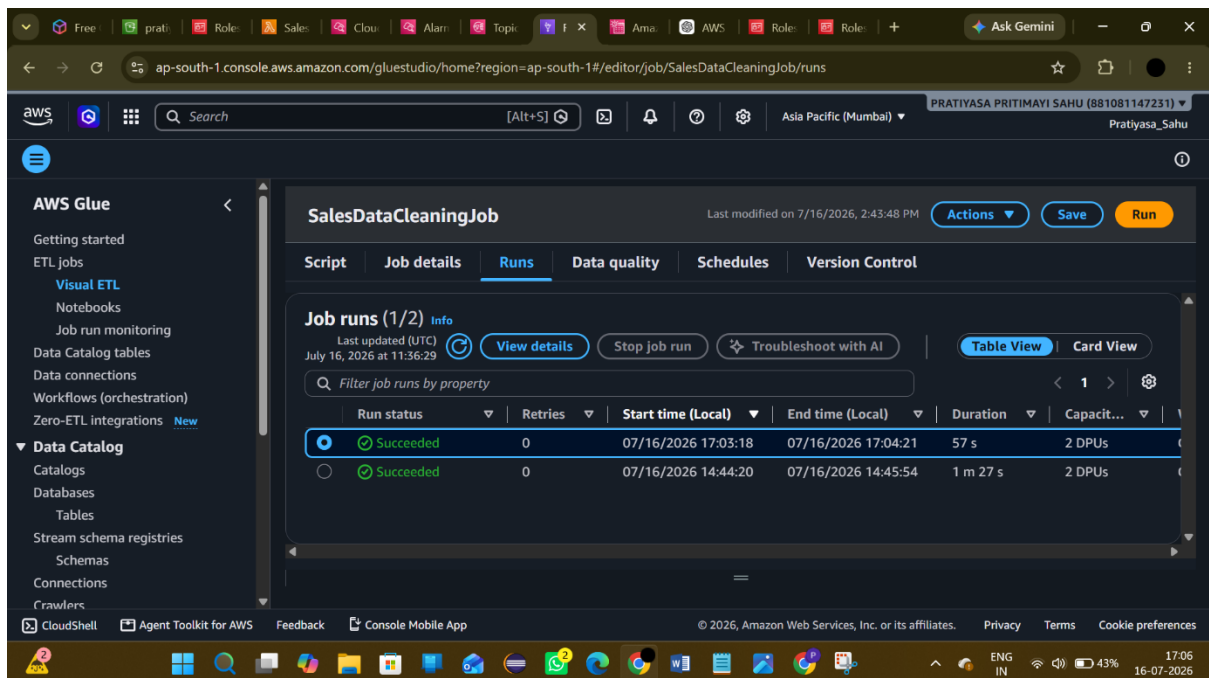
CloudWatch logs of lambda function copying the file to raw-zone S3 bucket and triggering the data cleaning glue job to start



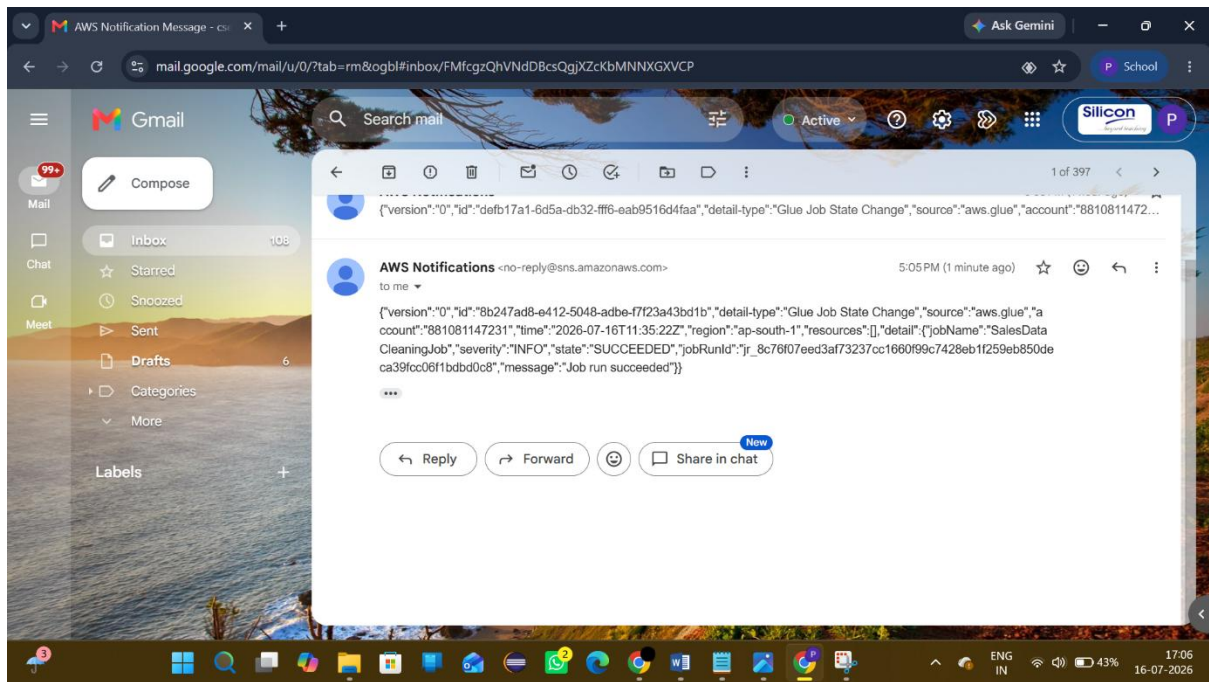
Sales.csv file copied to raw-zone bucket



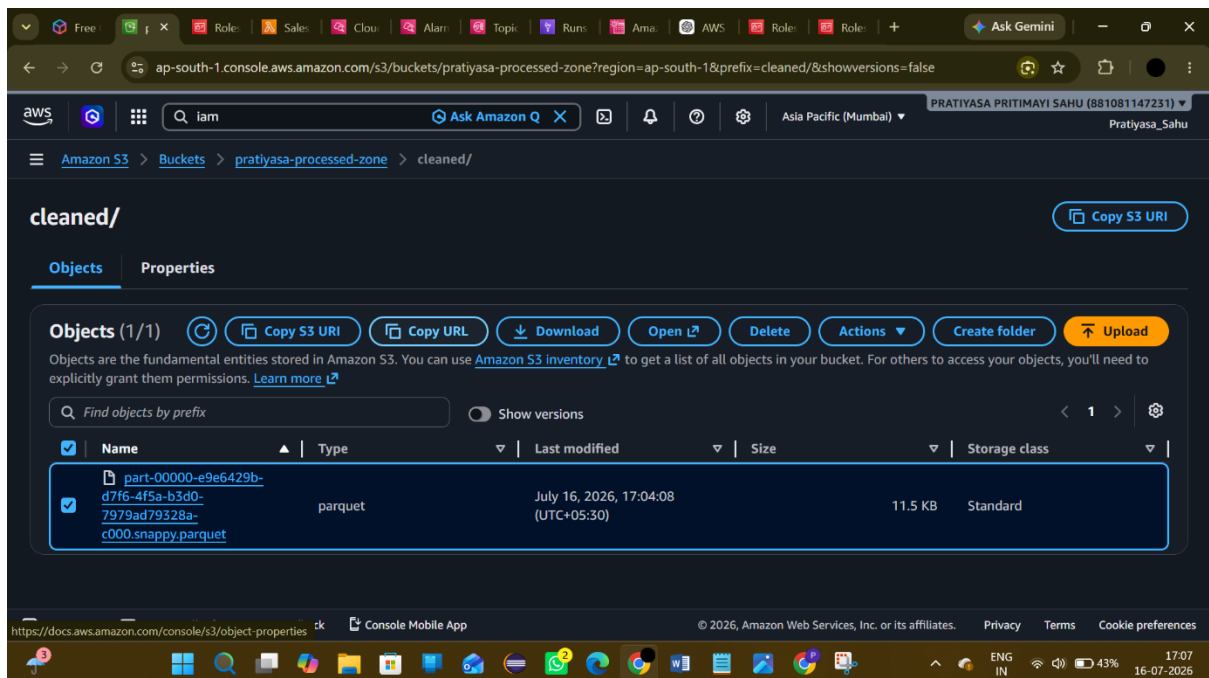
SaleCleaningJob triggered automatically and succeeded



Glue job success message received in selected email endpoint



Cleaned file dropped into processed-zone S3 bucket and converted to parquet format



CloudWatch logs of ETL Glue job success

The screenshot shows the AWS CloudWatch console interface. The left sidebar contains navigation options like 'CloudWatch', 'Log management', and 'Log Management'. The main panel displays 'Log events' for a specific log group. The log events are as follows:

Timestamp	Message
2026-07-16T17:03:50.940+05:30	===== ETL JOB STARTED ===== Reading CSV...
2026-07-16T17:04:03.883+05:30	Rows Before Cleaning : 1005
2026-07-16T17:04:05.991+05:30	Rows After Cleaning : 990 Writing Parquet...
2026-07-16T17:04:09.121+05:30	===== ETL JOB COMPLETED SUCCESSFULLY =====
2026-07-16T17:04:10.868+05:30	Running autoDebugger shutdown hook.

The interface also includes a search bar, a filter bar, and a 'Display' button. The bottom status bar shows the user's name 'Pratiyasa_Sahu' and the region 'Asia Pacific (Mumbai)'.

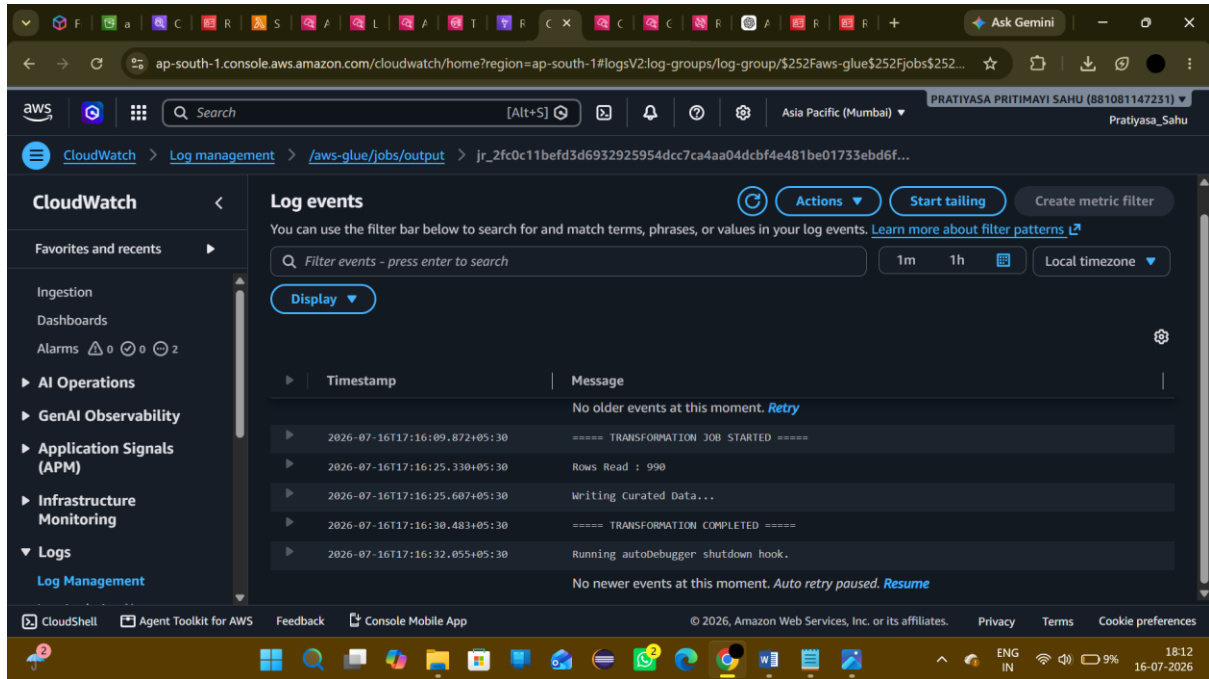
SalesTransformation job triggered at 5:15 PM and also succeeded

The screenshot shows the AWS Glue console interface. The left sidebar contains navigation options like 'AWS Glue', 'Getting started', 'ETL jobs', 'Visual ETL', 'Notebooks', 'Job run monitoring', 'Data Catalog tables', 'Data connections', 'Workflows (orchestration)', 'Zero-ETL integrations', 'Data Catalog', 'Catalogs', 'Databases', 'Tables', 'Stream schema registries', 'Schemas', 'Connections', and 'Crawlers'. The main panel displays the 'SalesTransformationJob' details. The 'Runs' tab is selected, showing a table of job runs.

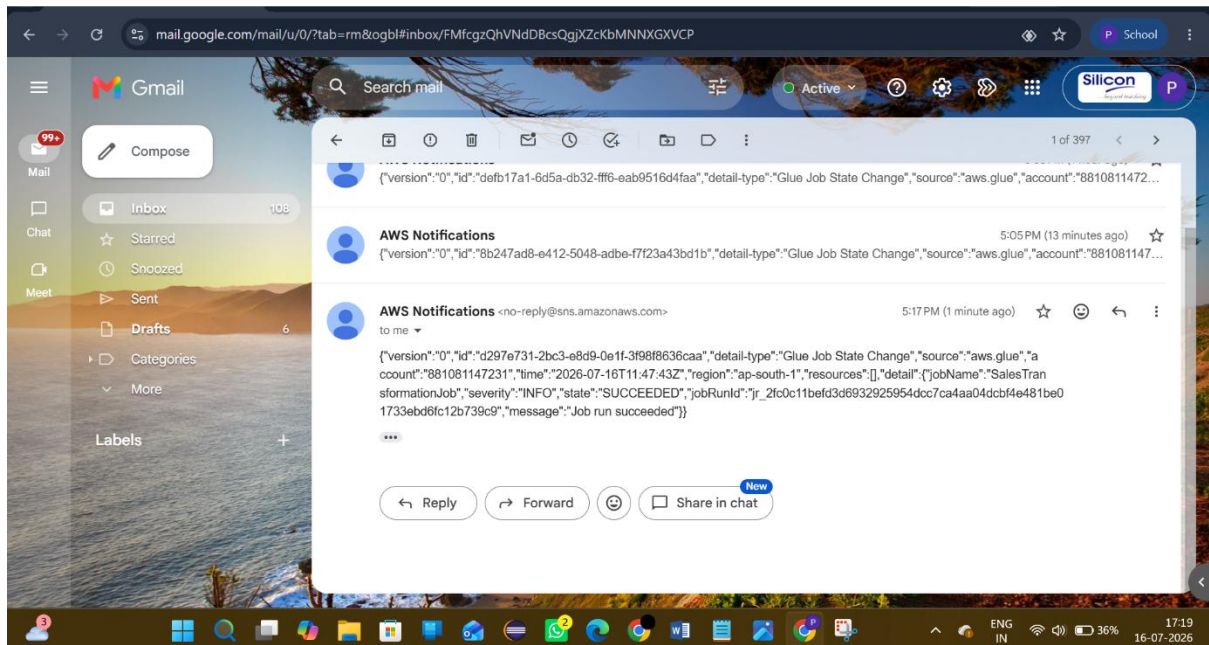
Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacit...
Succeeded	0	07/16/2026 17:15:29	07/16/2026 17:16:42	1 m 7 s	2 DPU's
Succeeded	0	07/16/2026 15:47:31	07/16/2026 15:48:40	1 m 12 s	2 DPU's

The interface also includes a search bar, a filter bar, and a 'Table View' button. The bottom status bar shows the user's name 'Pratiyasa_Sahu' and the region 'Asia Pacific (Mumbai)'.

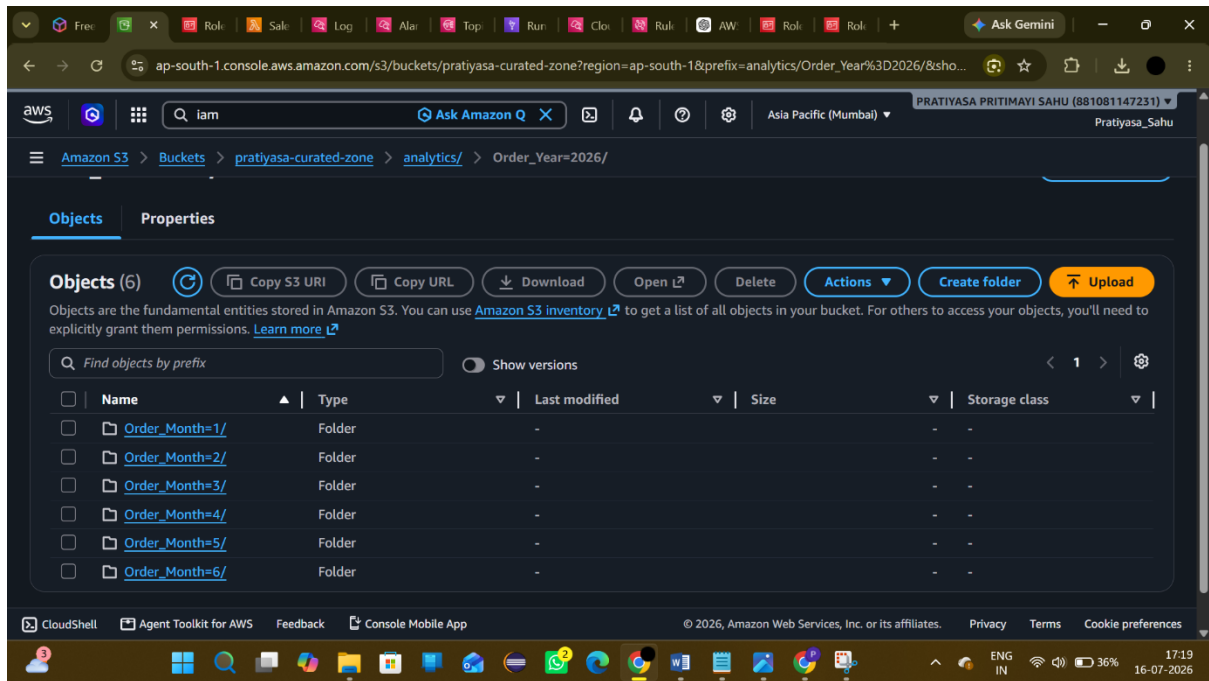
Cloudwatch logs of transformation glue job



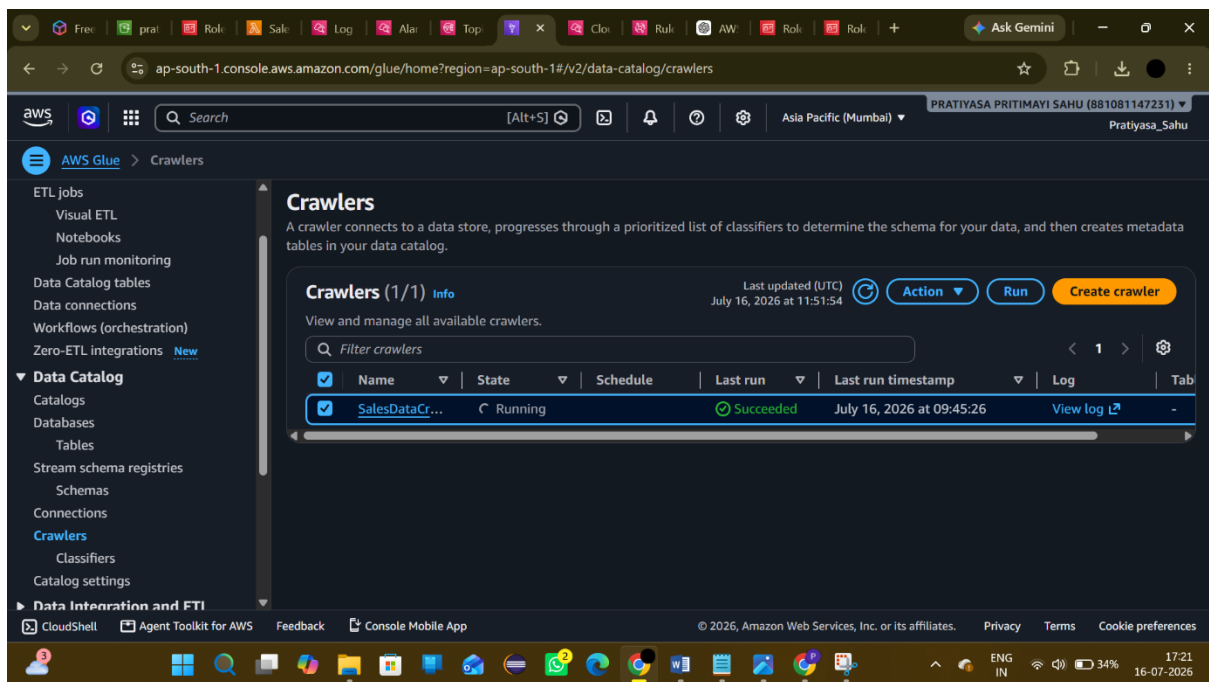
Glue job success message to email endpoint



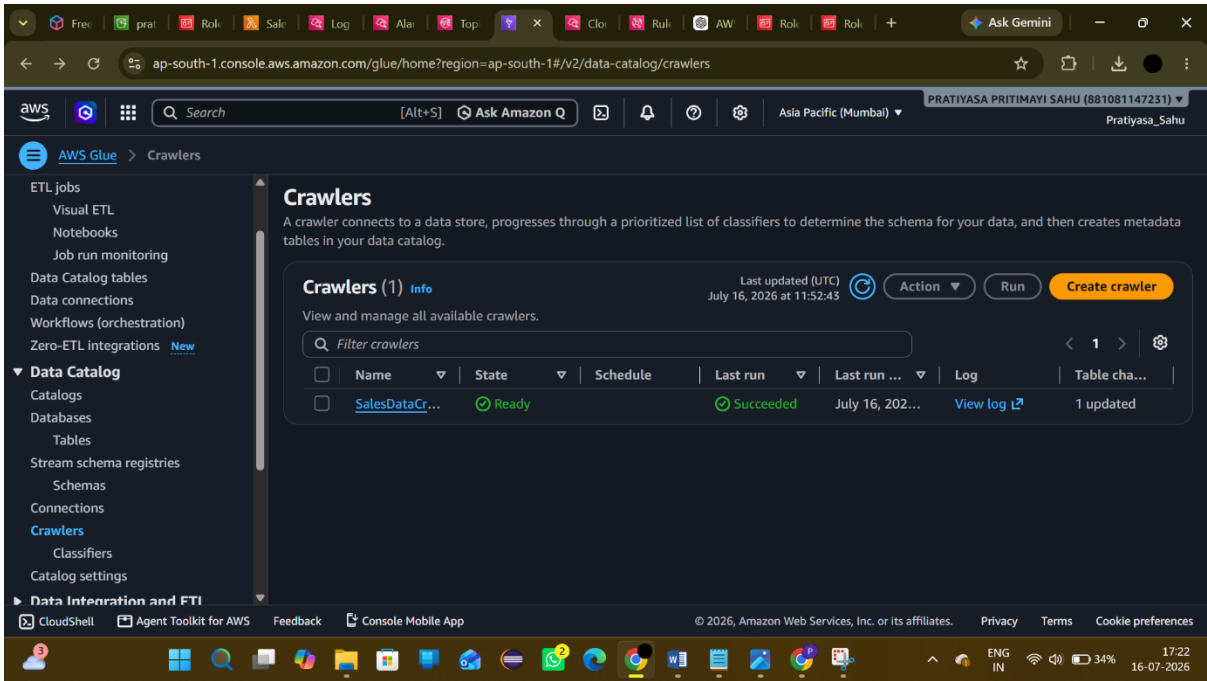
All processed and curated files in curated-zone S3 bucket



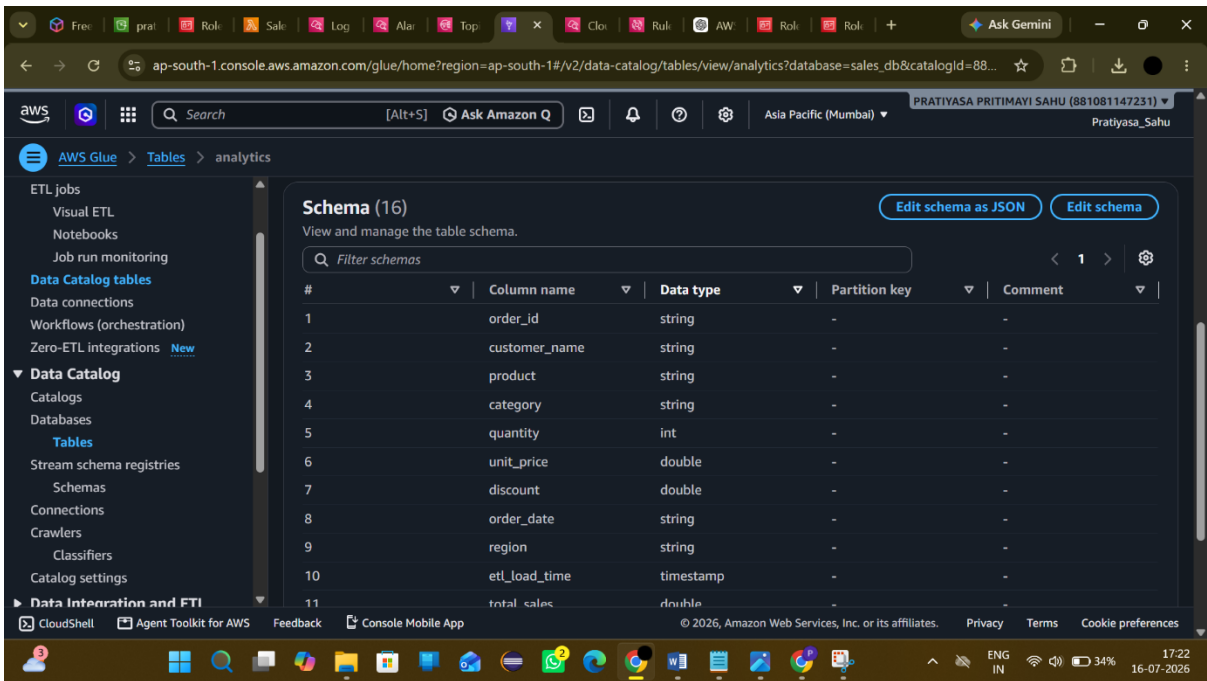
Glue Crawler running automatically at 5:20 PM automatically



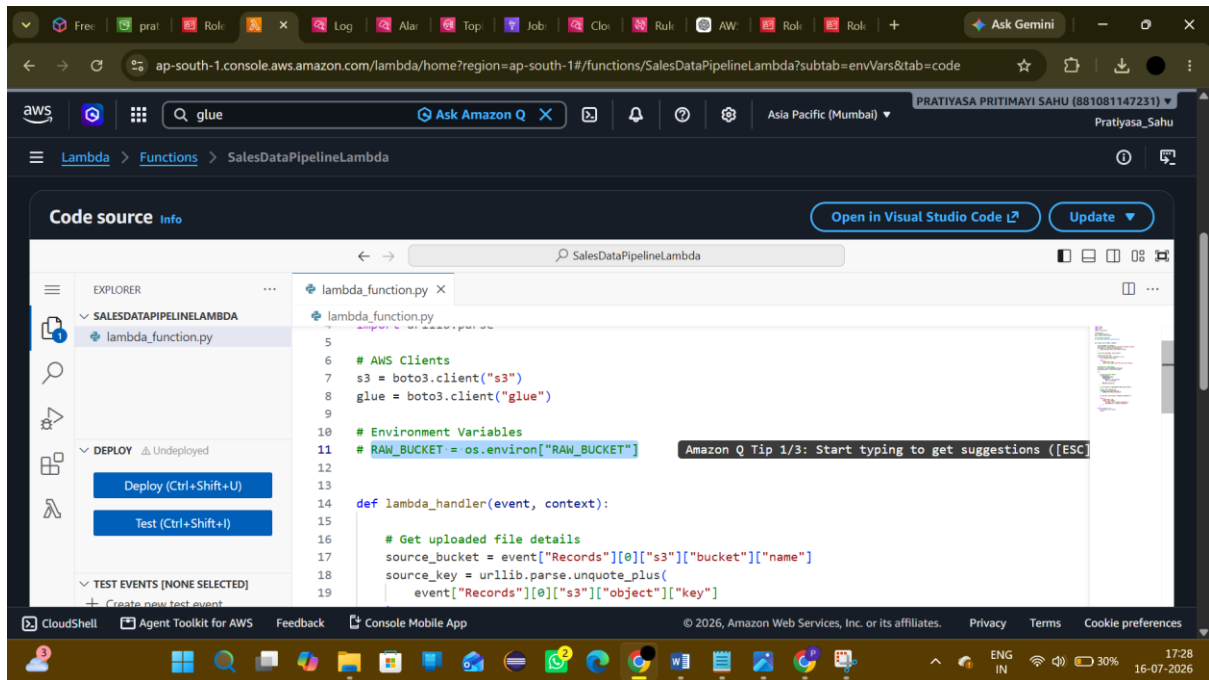
Crawler succeeded



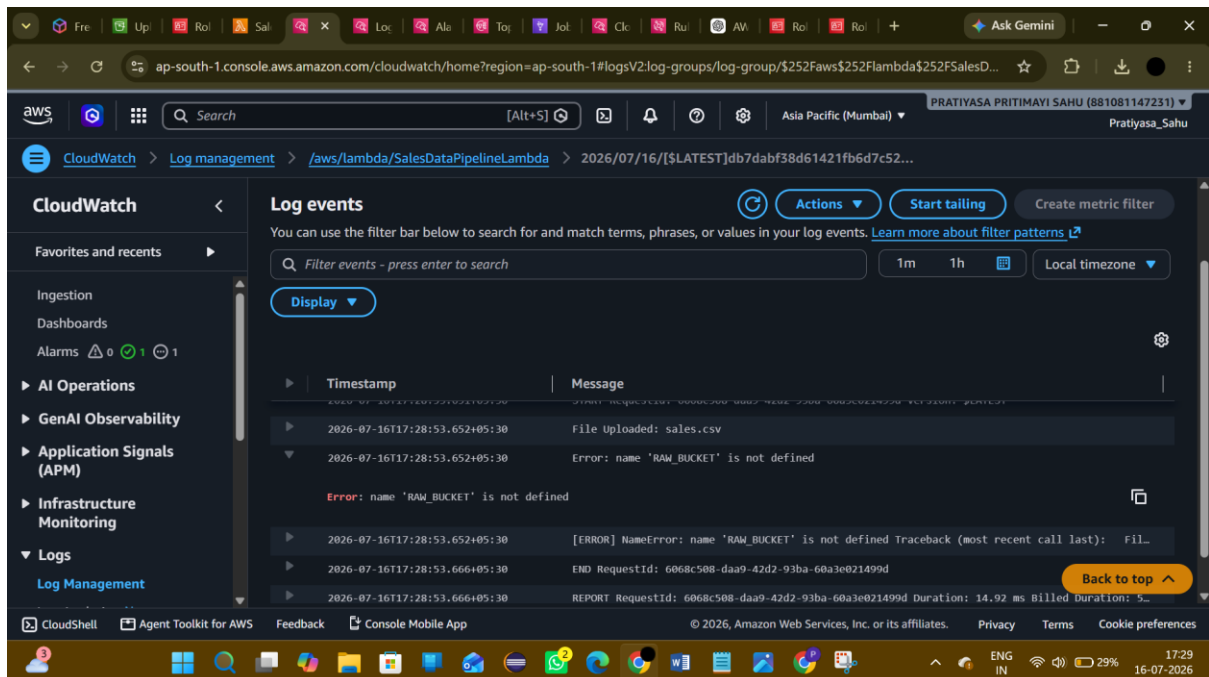
Schema of crawled database



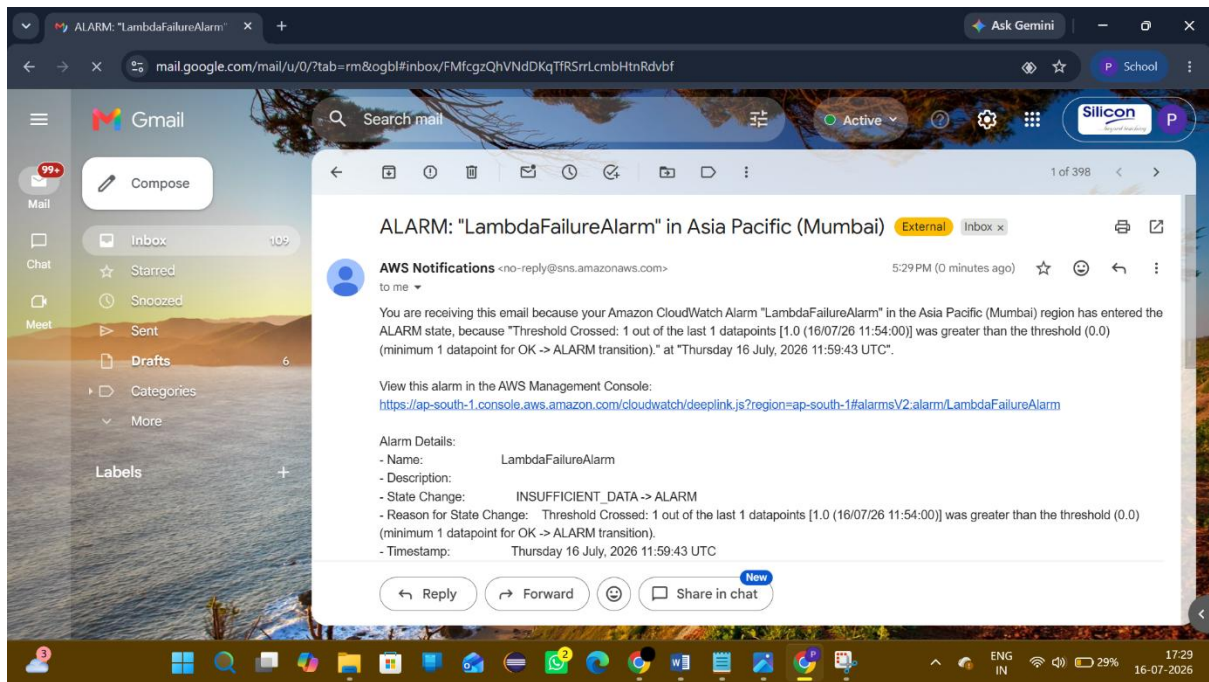
Making changes to lambda function to test lambda function failure



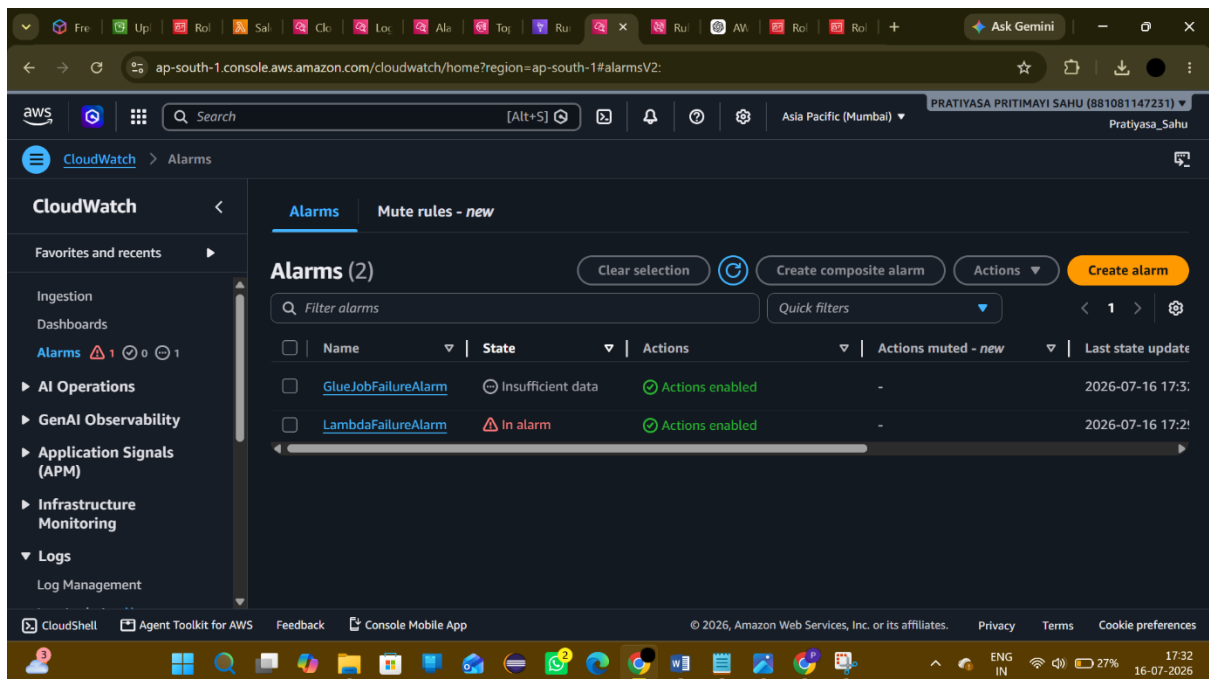
Lambda Function failed



Alarm notification of lambda failure



Alarm state triggered after failure



Making changes to cleaning data glue job to test failure state

The screenshot shows the AWS Glue console interface. The left sidebar contains the 'AWS Glue' menu with options like 'Getting started', 'ETL jobs', 'Visual ETL', 'Notebooks', 'Job run monitoring', 'Data Catalog tables', 'Data connections', 'Workflows (orchestration)', 'Zero-ETL integrations', 'Data Catalog', 'Catalogs', 'Databases', 'Tables', 'Stream schema registries', 'Schemas', 'Connections', and 'Crawlers'. The main panel displays the 'SalesDataCleaningJob' details, including a 'Script' tab with the following code:

```
24 # bucket paths
25 # -----
26
27 RAW_PATH = "s3://input/"
28
29 PROCESSED_PATH = "s3://pratiyasa-processed-zone/cleaned/"
30
31 try:
32
33 # -----
34 # Read CSV
```

The script is written in Python. The console shows the job was last modified on 7/16/2026 at 2:43:48 PM. The bottom status bar indicates the user is Pratiyasa_Sahu in the Asia Pacific (Mumbai) region.

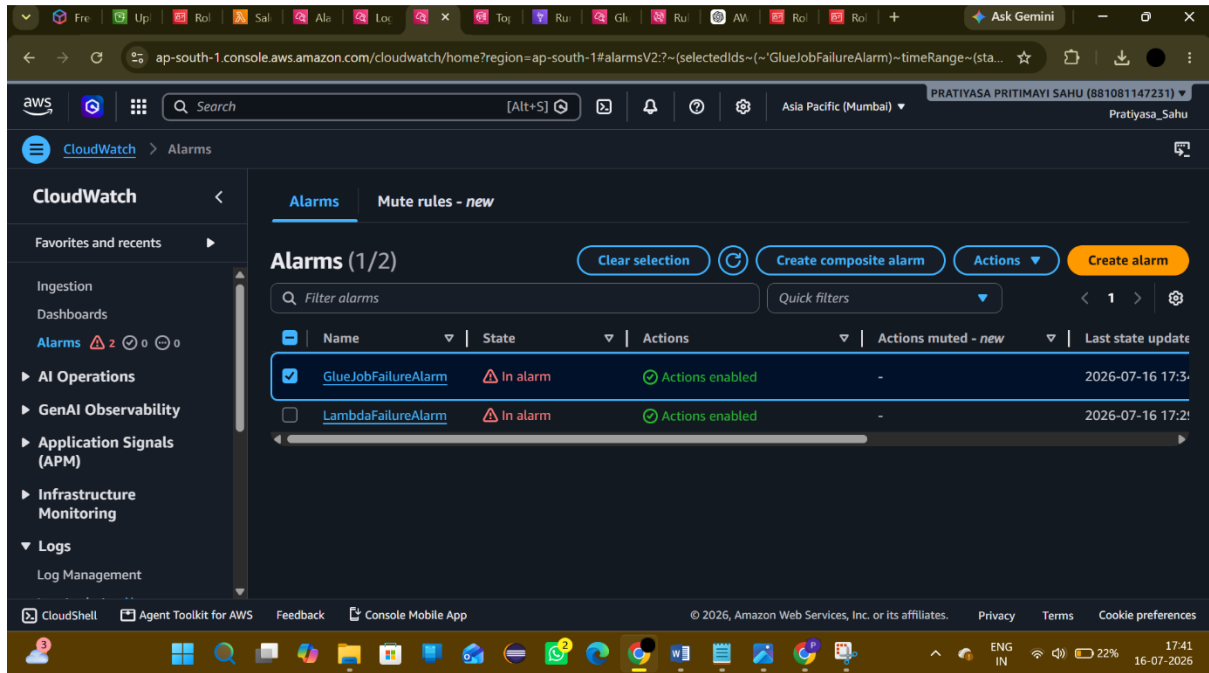
Job failed

The screenshot shows the AWS Glue console interface with the 'Runs' tab selected for the 'SalesDataCleaningJob'. The job runs are listed in a table:

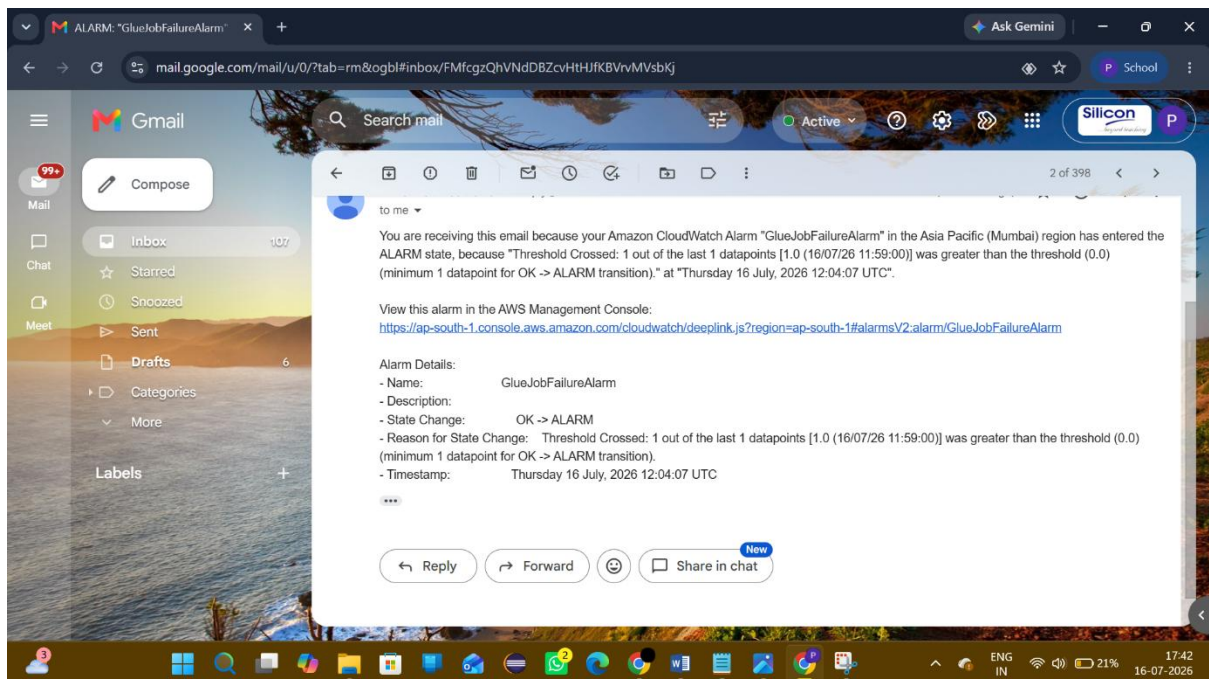
Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacit...
Failed	0	07/16/2026 17:38:17	07/16/2026 17:39:10	47 s	2 DPU's
Succeeded	0	07/16/2026 17:31:34	07/16/2026 17:33:14	1 m 6 s	2 DPU's

The 'Failed' run is highlighted. The console shows the job was last updated (UTC) on July 16, 2026 at 12:09:17. The bottom status bar indicates the user is Pratiyasa_Sahu in the Asia Pacific (Mumbai) region.

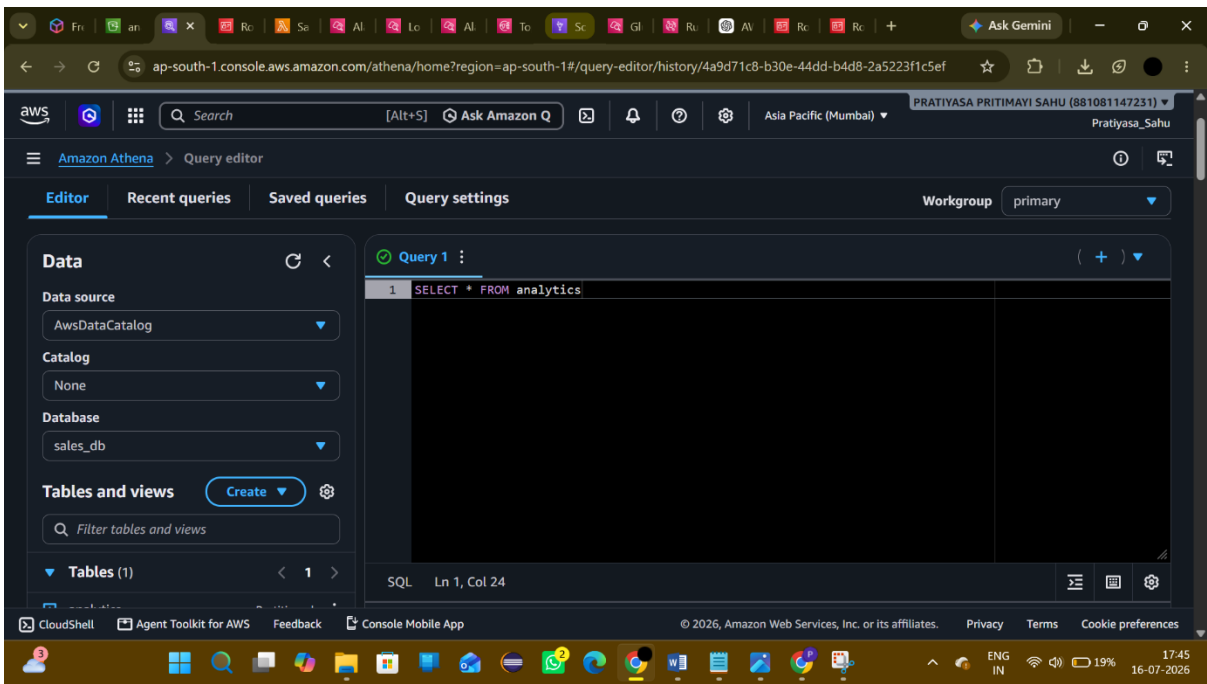
Alarm state triggered due to job failure



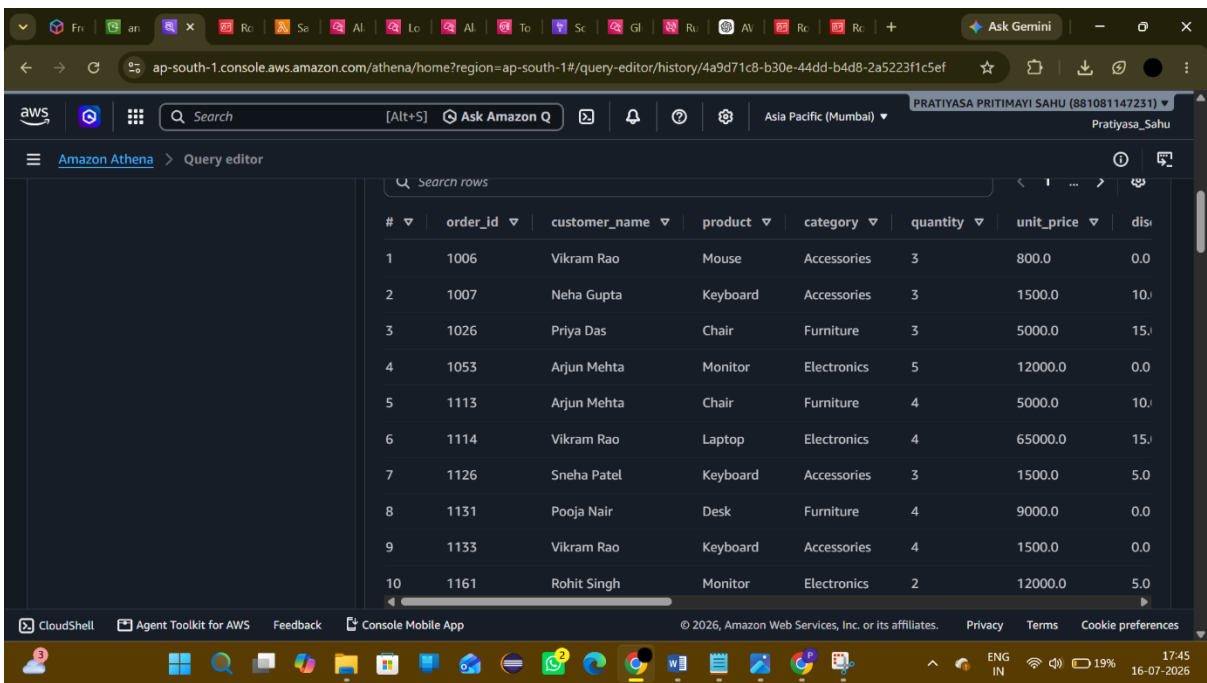
Email notification for glue job failure



Querying the final curated data through Athena



Result received of final curated data



More queries ran to the sales_db database

The screenshot shows the Amazon Athena Query Editor interface. On the left, the 'Data' panel is visible with the following settings: Data source: AwsDataCatalog, Catalog: None, Database: sales_db. Below this, the 'Tables and views' section shows a list of tables, including 'analytics'. The main query editor area contains the following SQL query:

```
1  
2 SELECT  
3 customer_name,  
4 SUM(final_sales) AS Revenue  
5 FROM analytics  
6 GROUP BY customer_name  
7 ORDER BY Revenue DESC  
8 LIMIT 5;
```

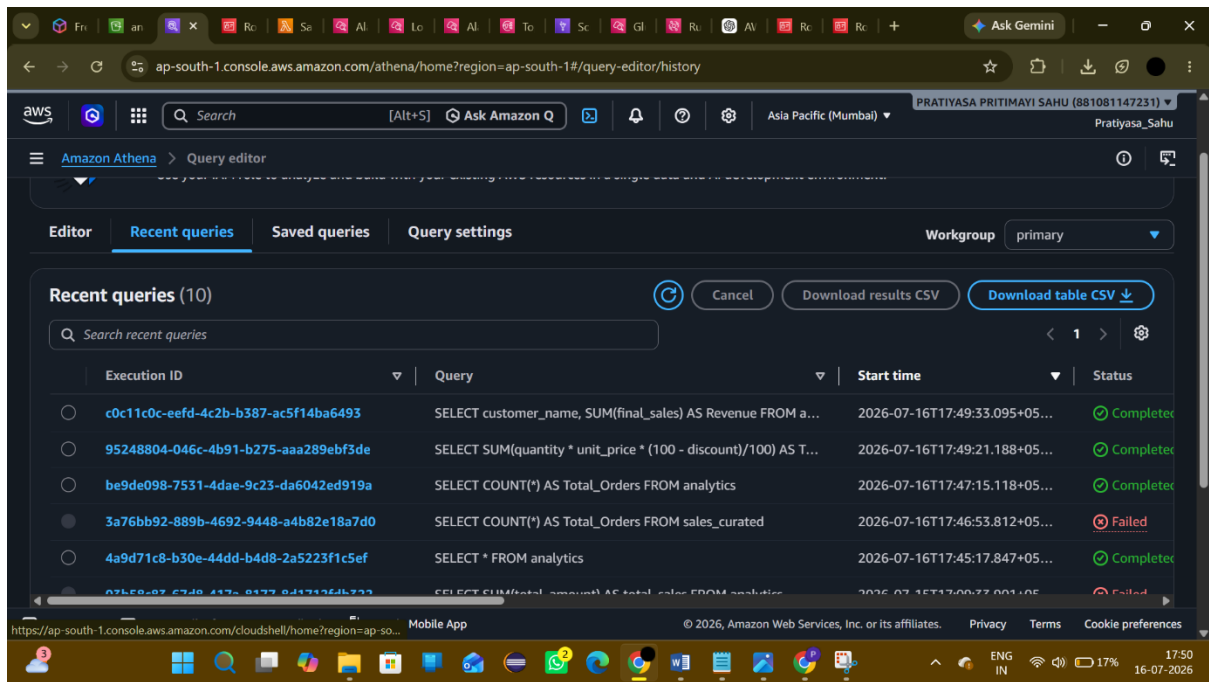
At the bottom of the query editor, there are buttons for 'Run again', 'Explain', 'Cancel', 'Clear', and 'Create'. The status bar at the bottom indicates 'SQL Ln 8, Col 9'.

Result/output of query

The screenshot shows the Amazon Athena Query Editor interface with the 'Query results' tab selected. The query has completed successfully. The status bar at the top of the results section indicates: 'Completed', 'Time in queue: 60 ms', 'Run time: 860 ms', and 'Data scanned: 5.61 KB'. Below this, the 'Results (5)' section displays a table with 5 rows of data. The table has columns for '#', 'customer_name', and 'Revenue'. The data is as follows:

#	customer_name	Revenue
1	Arjun Mehta	6069055.0
2	Vikram Rao	6011105.0
3	Pooja Nair	4771030.0
4	Amit Kumar	4210700.0
5	Sneha Patel	4125340.0

All query history



• Challenges Faced

Several challenges were encountered during the implementation of the project:

- Configuring IAM roles and permissions for Lambda, Glue, EventBridge Scheduler, and S3.
- Resolving Access Denied errors while invoking Glue jobs from Lambda.
- Handling Glue ETL script errors and debugging Spark syntax issues.
- Managing Glue Crawlers to correctly catalog Parquet datasets.
- Configuring EventBridge Scheduler to execute Glue Crawlers and Transformation Jobs automatically.
- Debugging CloudWatch logs to identify Glue execution failures.
- Setting up SNS notifications for Lambda and Glue job failures.
- Querying Parquet datasets using Amazon Athena after catalog creation.

• **Learning Outcomes**

Through this project, the following skills were gained:

- Designing an end-to-end serverless data engineering pipeline on AWS.
 - Automating ETL workflows using AWS Lambda and AWS Glue.
 - Working with Parquet files for efficient storage and analytics.
 - Managing metadata using AWS Glue Crawlers and Glue Data Catalog.
 - Querying large datasets using Amazon Athena.
 - Implementing scheduled workflows using Amazon EventBridge Scheduler.
 - Monitoring AWS resources using CloudWatch Logs and Alarms.
 - Configuring IAM roles and least-privilege permissions for secure service integration.
 - Implementing event-driven architectures using Amazon S3 triggers.
 - Integrating AWS services to build a scalable and automated data processing solution
-